

SELENIUM AUTOMATION & TESTING COMPREHENSIVE MASTER GUIDE

Complete Collection of 23 Core Topics: Architecture, WebDriver API, Waits, ActionChains, Window & Frame Handling, JS Execution, Network Logs & Exceptions

DOCUMENT SUMMARY

Total Modules Merged: 23 Docx Documents

Primary Domain: Selenium WebDriver (Python & Java)

Features Included: Full Code Snippets, Hierarchy, Waits & Exceptions

Generated Date: August 24, 2026

Selenium WebDriver Hierarchy

Understanding Selenium hierarchy is VERY important for:

- framework design
- interview questions
- advanced automation
- Selenium internals

1. Complete Selenium Driver Hierarchy

SearchContext



WebDriver



RemoteWebDriver



↑ ↑ ↑

ChromiumDriver FirefoxDriver SafariDriver



↑ ↑

ChromeDriver EdgeDriver

2. SearchContext (Top Level)

This is the root interface.

Provides:

- element searching capability

Methods:

find_element()

find_elements()

Example

element = driver.find_element(By.ID, "username")

Both:

- WebDriver
- WebElement

inherit from:

- SearchContext

Hierarchy

SearchContext



WebDriver



WebElement

That is why BOTH can search elements.

3. WebDriver Interface

Main Selenium interface.

Defines browser-level operations.

Common methods:

Method	Purpose
get()	Open URL
back()	Browser back
forward()	Browser forward
refresh()	Refresh page
close()	Close tab
quit()	Close browser
title	Get page title
current_url	Get URL

WebDriver Role

Defines WHAT browser should do.
Not HOW.

4. RemoteWebDriver

This is the actual core implementation.

Implements:

- WebDriver
- JavascriptExecutor
- TakesScreenshot
- Interactive
- HasCapabilities

Major Role of RemoteWebDriver

Handles:

- HTTP communication
- W3C protocol
- Selenium commands
- Session management
- Remote execution

Architecture

Test Script



RemoteWebDriver



HTTP Commands



Browser Driver



Browser

Important Point

Almost EVERYTHING in Selenium finally works through:

- RemoteWebDriver

5. ChromiumDriver

Intermediate class for Chromium browsers.

Parent of:

- ChromeDriver
- EdgeDriver

Provides Chromium-specific implementation.

Hierarchy

RemoteWebDriver



ChromiumDriver



ChromeDriver

Chromium Browsers

Browser Uses ChromiumDriver?

Chrome	Yes
Edge	Yes
Brave	Yes
Opera	Yes

6. ChromeDriver

Actual Chrome implementation.

Used in:

driver = webdriver.Chrome()

Internally:

ChromeDriver



ChromiumDriver



RemoteWebDriver



WebDriver

ChromeDriver Responsibilities

- Starts chromedriver.exe
- Creates session
- Launches Chrome
- Translates Selenium commands

7. FirefoxDriver

Separate implementation for Firefox.

RemoteWebDriver



FirefoxDriver

Uses:

- geckodriver

8. EdgeDriver

Edge also Chromium-based.

RemoteWebDriver



ChromiumDriver



EdgeDriver

Uses:

- msedgedriver

9. Browser Driver Role

Browser drivers act as translators.

Selenium Command Driver Translation

click()	Browser native action
send_keys()	Keyboard events
get()	Navigation command

Example Flow

driver.find_element(By.ID, "btn").click()

Internal flow:

Python Selenium



RemoteWebDriver



HTTP Request



ChromeDriver



Chrome DevTools Protocol



Chrome Browser

10. Browser Itself

Actual UI application:

- Chrome.exe
- firefox.exe
- msedge.exe

Selenium never directly controls browser.

It controls:

- browser driver

Driver controls:

- browser

11. Important Selenium Interfaces

A. JavascriptExecutor

Used for JavaScript execution.

`driver.execute_script()`

Implemented by:

- RemoteWebDriver

B. TakesScreenshot

`driver.save_screenshot()`

Implemented by:

- RemoteWebDriver

C. HasCapabilities

`driver.capabilities`

Provides:

- browser version
- platform
- capabilities

13. Most Important Interview Concept

Question:

Why is RemoteWebDriver important?

Answer:

RemoteWebDriver is the core Selenium implementation responsible for HTTP communication, session management, command execution, and browser interaction. All browser-specific drivers like ChromeDriver and FirefoxDriver ultimately inherit from it.

14. Super Important Hidden Concept

Even local execution is actually remote architecture internally.

When you run:

`webdriver.Chrome()`

internally:

Python Script

↓

RemoteWebDriver

↓ HTTP

ChromeDriver Server

↓

Chrome Browser

ChromeDriver itself is a small HTTP server.

15. Real-Time Enterprise Architecture

Pytest Framework

↓

Page Objects

↓

WebDriver Factory

↓

RemoteWebDriver

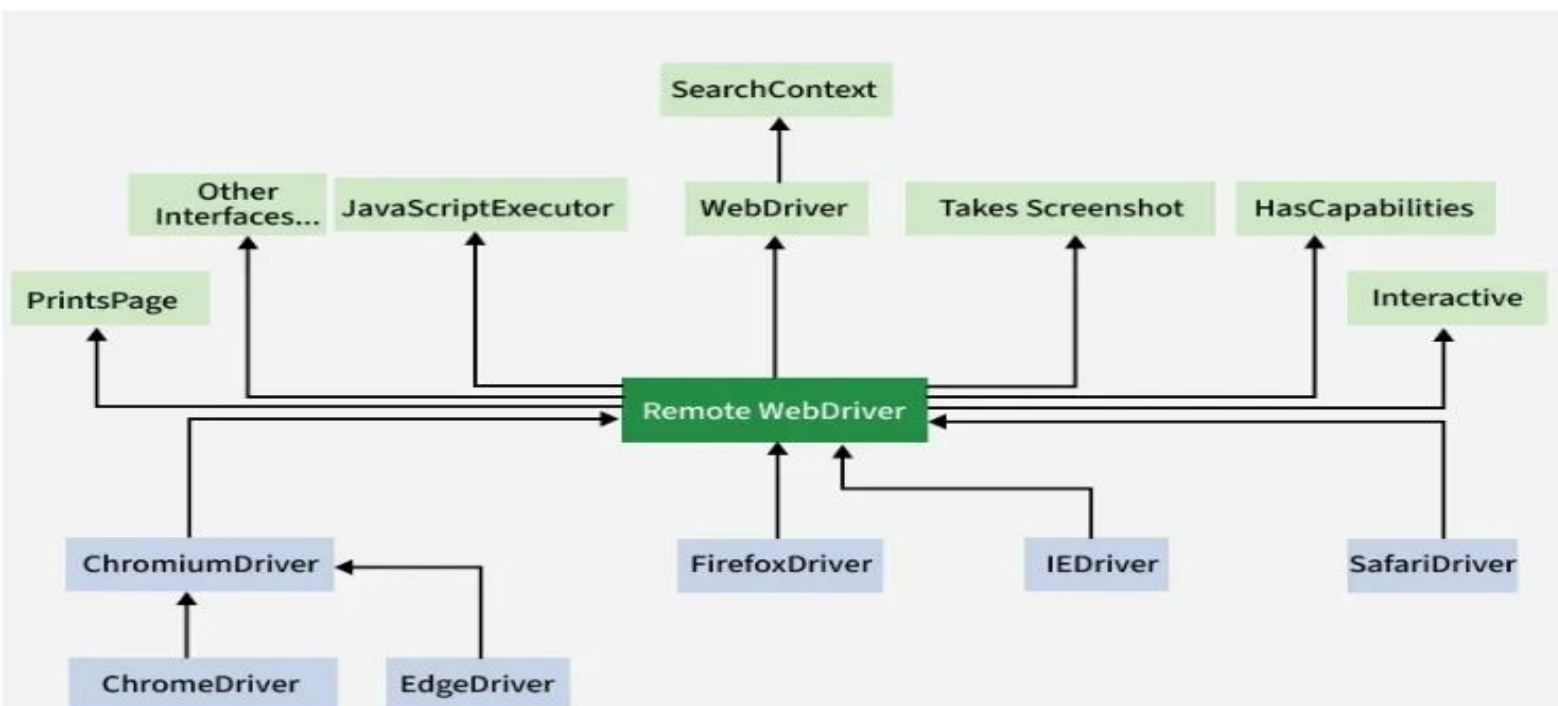
↓

Grid / Driver Server

↓

Browser

This is how enterprise frameworks work.



The main difference between WebDriver and RemoteWebDriver in Selenium is:

Feature	WebDriver	RemoteWebDriver
Execution	Runs locally	Runs remotely
Browser Location	Same machine	Another machine/server
Usage	Normal automation	Grid/Cloud/Distributed automation
Speed	Faster locally	Depends on network
Infrastructure	Single machine	Multiple machines/nodes
Common Use	Local testing	Selenium Grid, Docker, Cloud

1. WebDriver

WebDriver launches and controls the browser on your local system.

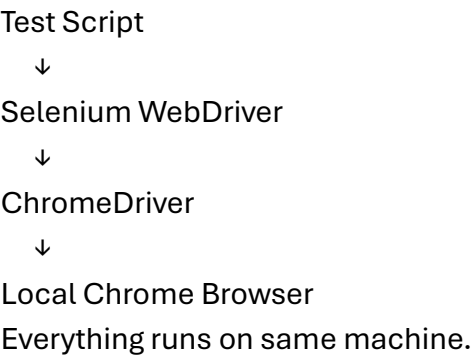
Example:
from selenium import webdriver

driver = webdriver.Chrome()

driver.get("https://google.com")

- Here:
- Chrome opens on your own machine
 - Selenium talks directly to local ChromeDriver

Architecture of Local WebDriver



2. RemoteWebDriver

RemoteWebDriver runs browser on another machine/server.

- Used in:
- Selenium Grid
 - Docker Selenium
 - BrowserStack
 - LambdaTest

- Sauce Labs
- Distributed execution

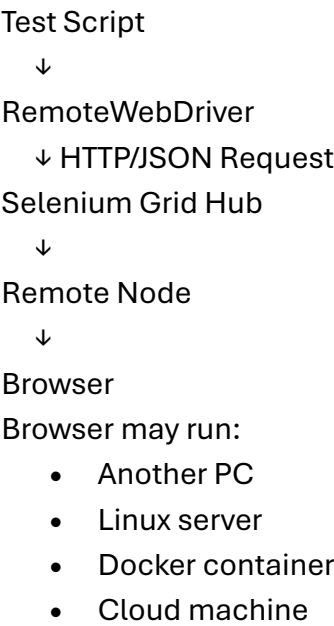
Example:
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities

options = Options()

driver = webdriver.Remote(
command_executor="http://localhost:4444/wd/hub",
options=options
)

driver.get("https://google.com")

Architecture of RemoteWebDriver



Key Difference

Local WebDriver

webdriver.Chrome()
Direct communication:

- Selenium → ChromeDriver → Chrome

RemoteWebDriver

webdriver.Remote()
Communication over network:

- Selenium → HTTP Request → Remote Server → Browser

Real-Time Example

WebDriver

You are driving your own car directly.

RemoteWebDriver

You send instructions to another driver remotely.

3. When We Use WebDriver

Used for:

- Local development
- Small projects
- Learning Selenium
- Single browser execution

Example:

- Running tests from laptop

4. When We Use RemoteWebDriver

Used for:

- Parallel execution
- Multiple browsers
- CI/CD pipelines
- Distributed testing
- Cloud execution

Example:

- Chrome + Firefox + Edge simultaneously
- Linux + Windows execution
- Docker containers
- Selenium Grid

5. Selenium Grid Example

Suppose:

Machine Role

Laptop Test execution

Server 1 Chrome node

Server 2 Firefox node

Your script runs locally but browsers execute remotely.

6. BrowserStack Example

```
driver = webdriver.Remote(
```

```
    command_executor="https://hub.browserstack.com/
wd/hub",
    options=options
)
```

Browser runs on BrowserStack cloud infrastructure.

7. RemoteWebDriver Advantages

Advantage	Description
Parallel execution	Run multiple tests simultaneously
Cross-browser testing	Chrome, Firefox, Edge
Cross-platform testing	Windows, Linux, Mac
CI/CD integration	Jenkins/GitLab
Scalability	Large enterprise automation
Docker support	Containerized execution

8. RemoteWebDriver Disadvantages

Issue	Reason
Network dependency	Uses HTTP communication
Slightly slower	Remote communication overhead
Infrastructure setup	Grid/Docker needed
Debugging complexity	Harder than local

9. DesiredCapabilities (Old Concept)

Earlier:
from
selenium.webdriver.common.desired_capabilities
import DesiredCapabilities

capabilities = DesiredCapabilities.CHROME.copy()
Now Selenium 4 prefers:

- Options()

Example:
options = webdriver.ChromeOptions()

```
driver = webdriver.Remote(
    command_executor="http://localhost:4444",
    options=options
)
```

10. Enterprise Framework Usage

In advanced frameworks:

- Local execution → webdriver.Chrome()
- CI/CD/Grid execution → webdriver.Remote()

Usually controlled from:

- config.ini
- pytest CLI
- Jenkins parameter
- environment variable

Example:

if execution_env == "local":

```
driver = webdriver.Chrome(options=options)
```

elif execution_env == "remote":

```
driver = webdriver.Remote(
    command_executor=grid_url,
    options=options
)
```

11. Interview Answer

What is the difference between WebDriver and RemoteWebDriver?

WebDriver controls browsers on the local machine directly using browser drivers like ChromeDriver. RemoteWebDriver controls browsers running on remote machines through network communication and is mainly used in Selenium Grid, Docker, and cloud testing platforms.

12. Important Technical Point

Actually:

webdriver.Chrome()

internally ALSO uses RemoteWebDriver.

ChromeDriver itself acts like a local server.

That means:

webdriver.Chrome()

↓

RemoteWebDriver

↓

Local ChromeDriver Server

So technically:

- ChromeDriver
- FirefoxDriver
- EdgeDriver

all inherit from:

- RemoteWebDriver

This is a very strong Selenium interview concept.

1. Setting Browser Using Options (Selenium 4 Recommended)

It understands by options import

Chrome Browser

```
from selenium import webdriver
```

```
from selenium.webdriver.chrome.options import Options
```

```
options = Options()
```

```
driver = webdriver.Remote(
    command_executor="http://localhost:4444",
    options=options
)
```

Here:

- ChromeOptions() tells Grid to open Chrome browser.

2. Firefox Browser

```
from selenium.webdriver.firefox.options import Options
```

```
from selenium import webdriver
```

```
options = Options()
```

```
driver = webdriver.Remote(
    command_executor="http://localhost:4444",
    options=options
)
```

Now Grid launches Firefox.

3. Edge Browser

```
from selenium.webdriver.edge.options import Options
```

```
options = Options()
```

```
driver = webdriver.Remote(
    command_executor="http://localhost:4444",
    options=options
)
```

Now Edge browser opens remotely.

How Selenium Understands Browser?

When you create:

```
ChromeOptions()
```

internally Selenium sends capability like:

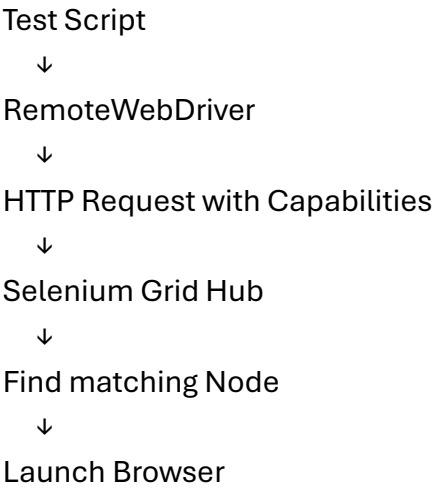
```
{
  "browserName": "chrome"
}
```

Similarly:

Options Class browserName Sent

ChromeOptions chrome
FirefoxOptions firefox
EdgeOptions MicrosoftEdge

Internal Flow



Cloud Credentials

Very important remote concept.
Example BrowserStack:
driver = webdriver.Remote(

command_executor="https://USERNAME:ACCESS_KEY@hub.browserstack.com/wd/hub",
options=options
)

14. What Is NOT Typically Seen in Local Driver?

Feature	Local	Remote
Grid URL	No	Yes
Cloud credentials	No	Yes
FileDetector	Rare	Common
Node routing	No	Yes
Cross-machine execution	No	Yes
Distributed session	No	Yes
Video recording	No	Yes
Network transport	Local only Remote HTTP	

15. Very Important Interview Concept

Question:

What extra capabilities does RemoteWebDriver provide?

Answer:

RemoteWebDriver provides distributed browser execution over HTTP and supports features such as Grid routing, cloud execution, desired capabilities, remote file upload using FileDetector, session management, remote command execution, and authenticated cloud execution using credentials.

16. One Advanced Internal Concept

Even this:
webdriver.Chrome()
internally creates:
RemoteConnection
RemoteWebDriver
HTTP Commands
But:

- communication stays on localhost.

So local Selenium is basically:

- a simplified remote execution model.

12. BrowserStack Example

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()

options.set_capability("browserName", "Chrome")
options.set_capability("browserVersion", "latest")

driver = webdriver.Remote(

command_executor="https://username:key@hub.browserstack.com/wd/hub",
options=options
)
```

Special remote things:

- credentials
- cloud URL
- capability routing

Below is a **table of commonly used Selenium Python methods** for both **WebDriver** and **WebElement**, with a short description for each.

 WebDriver Methods

Method	Description
driver.get(url)	Opens a webpage
driver.back()	Navigates to previous page
driver.forward()	Navigates to next page
driver.refresh()	Refreshes current page
driver.close()	Closes current browser tab
driver.quit()	Closes all browser windows and ends session
driver.maximize_window()	Maximizes browser window
driver.minimize_window()	Minimizes browser window
driver.fullscreen_window()	Opens browser in fullscreen mode
driver.find_element()	Finds single element
driver.find_elements()	Finds multiple elements
driver.execute_script()	Executes JavaScript
driver.execute_async_script()	Executes async JavaScript
driver.save_screenshot()	Takes screenshot
driver.get_screenshot_as_file()	Saves screenshot to file
driver.get_screenshot_as_png()	Returns screenshot as PNG bytes
driver.get_screenshot_as_base64()	Returns screenshot as base64
driver.implicitly_wait()	Sets implicit wait
driver.get_cookie()	Gets a cookie
driver.get_cookies()	Gets all cookies
driver.add_cookie()	Adds a cookie
driver.delete_cookie()	Deletes one cookie
driver.delete_all_cookies()	Deletes all cookies
driver.get_window_size()	Returns window size
driver.set_window_size()	Sets window size
driver.get_window_position()	Returns window position
driver.set_window_position()	Sets window position
driver.get_log()	Gets browser logs
driver.print_page()	Prints current page as PDF (Selenium 4)

 switch_to Methods

Method	Description
driver.switch_to.window()	Switches to another window
driver.switch_to.frame()	Switches to frame

Method	Description
driver.switch_to.parent_frame()	Switches to parent frame
driver.switch_to.default_content()	Switches to main page
driver.switch_to.new_window()	Opens new tab/window

 Alert Methods

Method	Description
alert.accept()	Clicks OK
alert.dismiss()	Clicks Cancel
alert.send_keys()	Types text in alert
alert.text	Reads alert message (property)

 WebElement Methods

Method	Description
element.click()	Clicks element
element.clear()	Clears input field
element.send_keys()	Enters text
element.submit()	Submits form
element.get_attribute()	Gets attribute value
element.get_dom_attribute()	Gets DOM attribute
element.get_property()	Gets property value
element.is_displayed()	Checks visibility
element.is_enabled()	Checks enabled state
element.is_selected()	Checks selected state
element.find_element()	Finds child element
element.find_elements()	Finds child elements
element.screenshot()	Takes element screenshot
element.value_of_css_property()	Gets CSS value

 ActionChains Common Methods

Method	Description
click()	Clicks element
double_click()	Double click
context_click()	Right click
move_to_element()	Mouse hover
drag_and_drop()	Drag and drop
click_and_hold()	Hold mouse
release()	Release mouse

Method	Description
send_keys()	Keyboard action
perform()	Executes action

✔ Select Class Methods

Used for dropdowns.

Method	Description
select_by_visible_text()	Select by visible text
select_by_index()	Select by index
select_by_value()	Select by value
deselect_by_visible_text()	Deselect option
deselect_by_index()	Deselect by index
deselect_by_value()	Deselect by value
deselect_all()	Deselect all options

✔ Most Frequently Used in Projects

Most important methods:

driver.get()

driver.find_element()

driver.find_elements()

driver.quit()

element.click()

element.send_keys()

element.get_attribute()

element.is_displayed()

✔ Example

```
driver.get("https://example.com")
```

```
username = driver.find_element(By.ID, "username")
```

```
username.send_keys("admin")
```

```
login_btn = driver.find_element(By.ID, "login")
```

```
login_btn.click()
```

If you'd like, I can prepare a **single interview-ready cheat sheet of Selenium Python methods + properties** for quick revision.

Below is the **Selenium Python properties list in table format with description** for easier understanding.

 WebDriver Properties

Property	Description
driver.title	Returns the title of the current webpage
driver.current_url	Returns the current page URL
driver.page_source	Returns complete HTML source of the page
driver.name	Returns browser name such as chrome or firefox
driver.session_id	Returns unique browser session ID
driver.capabilities	Returns browser capabilities/settings
driver.window_handles	Returns list of all open window handles
driver.current_window_handle	Returns current active window handle
driver.switch_to	Used to switch to alert, frame, or window

 switch_to Related Properties

Property	Description
driver.switch_to.alert	Returns current alert object
driver.switch_to.active_element	Returns currently focused element

 WebElement Properties

Property	Description
element.text	Returns visible text of element
element.tag_name	Returns HTML tag name of element
element.size	Returns element width and height
element.location	Returns x and y coordinates
element.rect	Returns size + location in one dictionary
element.accessible_name	Returns accessibility name
element.aria_role	Returns ARIA role of element
element.screenshot_as_png	Returns screenshot bytes of element

✔ Alert Property

Property Description

alert.text	Returns text message from alert popup
------------	---------------------------------------

✔ Example

```
print(driver.title)
```

```
print(driver.current_url)
```

```
print(element.text)
```

✔ Example Output

Login Page

https://example.com/login

Submit

✔ Quick Rule

Type	Example	Parentheses
Property	driver.title	No
Method	driver.quit()	Yes

If you'd like, I can create the **same table for all Selenium WebElement methods** like click(), send_keys(), clear() and more for interview preparation.

Priority wise should be in the list **[id, name, linkText, partialLinkText, tagName, className, cssSelector, xpath]**

Here is a cleaner, structured, interview-friendly, and easy-to-understand version of Selenium Python methods for your automation framework learning and interview preparation.

Selenium Python Methods Cheat Sheet (WebDriver + WebElement)

1. WebDriver Methods

WebDriver methods are used to control the browser.

Browser Navigation Methods

Method	Purpose
driver.get(url)	Opens a website URL
driver.back()	Goes to previous page
driver.forward()	Goes to next page
driver.refresh()	Refreshes current page

Browser Close Methods

Method	Purpose
driver.close()	Closes current tab/window
driver.quit()	Closes all browser windows and ends WebDriver session

Browser Window Methods

Method	Purpose
driver.maximize_window()	Maximizes browser
driver.minimize_window()	Minimizes browser
driver.fullscreen_window()	Opens fullscreen mode

Element Finding Methods

Method	Purpose
driver.find_element()	Finds single element
driver.find_elements()	Finds multiple elements

JavaScript Methods

Method	Purpose
driver.execute_script()	Executes JavaScript
driver.execute_async_script()	Executes asynchronous JavaScript

Screenshot Methods

Method	Purpose
driver.save_screenshot()	Takes screenshot

Method	Purpose
driver.get_screenshot_as_file()	Saves screenshot as file
driver.get_screenshot_as_png()	Returns screenshot as PNG bytes
driver.get_screenshot_as_base64()	Returns screenshot as Base64 string

Wait Method

Method	Purpose
driver.implicitly_wait()	Sets implicit wait

Cookie Methods

Method	Purpose
driver.get_cookie()	Gets single cookie
driver.get_cookies()	Gets all cookies
driver.add_cookie()	Adds cookie
driver.delete_cookie()	Deletes single cookie
driver.delete_all_cookies()	Deletes all cookies

Window Size & Position Methods

Method	Purpose
driver.get_window_size()	Returns browser size
driver.set_window_size()	Sets browser size
driver.get_window_position()	Returns browser position
driver.set_window_position()	Sets browser position

Browser Logs

Method	Purpose
driver.get_log()	Gets browser logs

Selenium 4 Method

Method	Purpose
driver.print_page()	Saves current page as PDF

2. switch_to Methods

Used for handling:

- windows
- tabs
- frames
- alerts

Method	Purpose
driver.switch_to.window()	Switches window/tab
driver.switch_to.frame()	Switches iframe
driver.switch_to.parent_frame()	Switches to parent frame
driver.switch_to.default_content()	Returns to main page
driver.switch_to.new_window()	Opens new tab/window

3. Alert Methods

Used for JavaScript alerts.

Method	Purpose
alert.accept()	Clicks OK
alert.dismiss()	Clicks Cancel
alert.send_keys()	Types text into alert
alert.text	Reads alert message

4. WebElement Methods

WebElement methods are used to interact with HTML elements.

Basic Interaction Methods	
Method	Purpose
element.click()	Clicks element
element.clear()	Clears text field
element.send_keys()	Types text
element.submit()	Submits form

Attribute & Property Methods

Method	Purpose
element.get_attribute()	Gets HTML attribute
element.get_dom_attribute()	Gets DOM attribute
element.get_property()	Gets JavaScript property

State Checking Methods

Method	Purpose
element.is_displayed()	Checks visibility
element.is_enabled()	Checks enabled state
element.is_selected()	Checks selected state

Child Element Methods

Method	Purpose
element.find_element()	Finds child element
element.find_elements()	Finds child elements

Screenshot Method

Method	Purpose
element.screenshot()	Takes element screenshot

CSS Property Method

Method	Purpose
element.value_of_css_property()	Gets CSS property value

5. ActionChains Methods

Used for advanced mouse and keyboard actions. Use .perform() in the last of every script

Import	
from selenium.webdriver.common.action_chains	
import ActionChains	

Method	Purpose
click()	Single click
double_click()	Double click
context_click()	Right click
move_to_element()	Mouse hover
drag_and_drop()	Drag and drop
click_and_hold()	Hold mouse button
release()	Release mouse
send_keys()	Keyboard typing
perform()	Executes all actions

6. Select Class Methods

Used for handling dropdowns.

Import

```
from selenium.webdriver.support.ui import Select
```

Method	Purpose
select_by_visible_text()	Select using visible text
select_by_index()	Select using index
select_by_value()	Select using value
deselect_by_visible_text()	Deselect by text
deselect_by_index()	Deselect by index
deselect_by_value()	Deselect by value
deselect_all()	Deselect all options

7. Most Frequently Used Methods in Real Projects

These methods are heavily used in automation frameworks.

WebDriver Methods

```
driver.get()
driver.find_element()
driver.find_elements()
driver.quit()
```

WebElement Methods

```
element.click()
element.send_keys()
element.get_attribute()
element.is_displayed()
```

8. Real-Time Example

```
from selenium.webdriver.common.by import By
```

```
driver.get("https://example.com")
```

```
username = driver.find_element(By.ID, "username")
username.send_keys("admin")
```

```
password = driver.find_element(By.ID, "password")
password.send_keys("123456")
```

```
login_btn = driver.find_element(By.ID, "login")
login_btn.click()
```

9. Interview Tips

Difference Between close() and quit()

Method Difference

```
close()  Closes current tab only
quit()   Closes entire browser session
```

Difference Between find_element and find_elements

Method	Difference
find_element()	Returns single WebElement
find_elements()	Returns list of WebElements

Difference Between get_attribute and get_property

Method	Difference
get_attribute()	Reads HTML attribute
get_property()	Reads JavaScript property

10. Best Practice

- Prefer:
- Explicit Waits
 - Page Object Model (POM)
 - Reusable utility methods

Avoid:
time.sleep()
unless absolutely necessary.

Interview Summary

Selenium Python provides WebDriver methods for browser control and WebElement methods for interacting with page elements. Additional classes like ActionChains, Select, Alerts, and switch_to are used for advanced user interactions, dropdown handling, popups, frames, and multi-window operations. These methods form the core of Selenium automation frameworks.

Here is a cleaner, structured, and interview-friendly version of Selenium Python Properties for WebDriver, WebElement, switch_to, and Alert handling.

Selenium Python Properties Cheat Sheet

Properties are used to retrieve information from:

- browser
- webpage
- elements
- alerts
- windows
- sessions

Unlike methods, properties are accessed without parentheses ().

Example:
driver.title
element.text

1. WebDriver Properties

WebDriver properties provide information about:

- browser
- webpage
- session
- windows

Page Information Properties

Property	Purpose
driver.title	Returns current webpage title
driver.current_url	Returns current webpage URL
driver.page_source	Returns complete HTML source code

Browser Information Properties

Property	Purpose
driver.name	Returns browser name (chrome/firefox)
driver.capabilities	Returns browser capabilities/settings

Session Properties

Property	Purpose
driver.session_id	Returns unique WebDriver session ID

Window Handling Properties

Property	Purpose
driver.window_handles	Returns list of all opened windows/tabs
driver.current_window_handle	Returns current active window handle

switch_to Property

Property	Purpose
driver.switch_to	Used for switching alerts, frames, and windows

2. switch_to Related Properties

Used for handling:

- alerts
- focused elements

Alert Property

Property	Purpose
driver.switch_to.alert	Returns current alert object

Active Element Property

Property	Purpose
driver.switch_to.active_element	Returns currently focused element

3. WebElement Properties

WebElement properties are used to retrieve information from HTML elements.

Text & Tag Properties

Property	Purpose
element.text	Returns visible text of element
element.tag_name	Returns HTML tag name

Size & Position Properties

Property	Purpose
element.size	Returns width and height
element.location	Returns x and y coordinates
element.rect	Returns size + position together

Accessibility Properties (Selenium 4)

Property	Purpose
element.accessible_name	Returns accessibility name
element.aria_role	Returns ARIA role

Screenshot Property

Property	Purpose
element.screenshot_as_png	Returns screenshot as PNG bytes

4. Alert Properties

Used for JavaScript alerts.

Property Purpose

alert.text Returns alert popup message

5. Real-Time Examples

Get Page Title

print(driver.title)

Get Current URL

print(driver.current_url)

Get HTML Source

print(driver.page_source)

Get Browser Name

print(driver.name)

Get Session ID

print(driver.session_id)

Get Window Handles

print(driver.window_handles)

Get Current Window Handle

print(driver.current_window_handle)

WebElement Examples

Get Element Text

print(element.text)

Get Tag Name

print(element.tag_name)

Get Element Size

print(element.size)

Example Output:

{'height': 40, 'width': 120}

Get Element Location

print(element.location)

Example Output:

{'x': 250, 'y': 300}

Get Element Rectangle

print(element.rect)

Example Output:

{
 'height': 40,
 'width': 120,
 'x': 250,

'y': 300

}

Get Accessibility Name

print(element.accessible_name)

Get ARIA Role

print(element.aria_role)

Alert Example

alert = driver.switch_to.alert

print(alert.text)

alert.accept()

Active Element Example

active_element = driver.switch_to.active_element

print(active_element.tag_name)

Difference Between Methods and Properties

Methods	Properties
Use parentheses ()	No parentheses
Perform actions	Return information
Example: click()	Example: text

Most Frequently Used Properties in Real Projects

driver.title
driver.current_url
driver.window_handles

element.text
element.tag_name
element.location

alert.text

Best Practice

Use properties for:

- validations
- assertions
- logging
- debugging
- reporting

Example:

assert "Dashboard" in driver.title

Interview Summary

Selenium properties are used to retrieve information from the browser, webpage, alerts, windows, and web elements.

WebDriver properties provide browser/session details, while WebElement properties provide text, size, position, and accessibility information. Properties are commonly used for validation, assertions, logging, debugging, and reporting in automation frameworks.

Yes. Based on the file you shared, the Selenium commands are divided into categories such as **Browser, Navigation, Window, Frames/IFrames, Alerts, Element Interaction, Element Actions, Waits, Keyboard/Mouse, Cookies, Screenshots, Session, Browser Options, Logs, JavaScript, File Upload/Download, Proxy, Remote WebDriver, and Timeouts.**

Since you specifically want this in **Python + Selenium + pytest**, here is a much more useful interview-revision version.

Python Selenium + Pytest — Selenium Commands by Category

1. Browser & Navigation

```
driver.get("https://example.com")
```

```
driver.back()
driver.forward()
driver.refresh()
```

```
driver.current_url
driver.title
```

```
driver.maximize_window()
driver.minimize_window()
driver.fullscreen_window()
```

```
driver.quit()
driver.close()
```

Important for interview

```
driver.get()
driver.back()
driver.forward()
driver.refresh()
driver.current_url
driver.title
driver.quit()
```

2. Locators / Finding Elements

Your source covers locating elements using ID, name, class, tag, link text, CSS and XPath.

Python

```
from selenium.webdriver.common.by import By
```

```
driver.find_element(By.ID, "username")
```

```
driver.find_element(By.NAME, "username")
```

```
driver.find_element(By.CLASS_NAME, "login-button")
```

```
driver.find_element(By.TAG_NAME, "input")
```

```
driver.find_element(By.LINK_TEXT, "Login")
```

```
driver.find_element(By.PARTIAL_LINK_TEXT, "Log")
```

```
driver.find_element(By.CSS_SELECTOR,
"#username")
```

```
driver.find_element(By.XPATH,
"//input[@id='username']")
```

Multiple elements

```
elements = driver.find_elements(
    By.XPATH,
    "//input"
)
```

```
print(len(elements))
```

Interview priority ★★★★★

Know these extremely well:

ID

NAME

CSS

XPATH

CLASS_NAME

find_element()

find_elements()

3. Element Actions

The source includes click, send keys, clear, submit, attributes, text, displayed/enabled/selected and JavaScript actions.

```
element = driver.find_element(By.ID, "username")
```

```
element.click()
```

```
element.send_keys("Neeraj")
```

```
element.clear()
```

```
element.submit()
```

```
element.text
```

```
element.get_attribute("value")
```

```
element.is_displayed()
```

```
element.is_enabled()
```

```
element.is_selected()
```

4. Waits ★★★★★

This is one of the **most important Selenium topics for interviews**.

Your source specifically includes visibility, clickability, URL, text, invisibility, staleness, alerts and iframe waits.

Explicit wait

```
from selenium.webdriver.support.ui import
WebDriverWait
from selenium.webdriver.support import
expected_conditions as EC
```

```
wait = WebDriverWait(driver, 10)
```

```
element = wait.until(
    EC.visibility_of_element_located(
        (By.ID, "username")
    )
)
```

Element clickable

```
element = wait.until(
    EC.element_to_be_clickable(
        (By.ID, "login")
    )
)
```

```
element.click()
```

Presence

```
wait.until(
    EC.presence_of_element_located(
        (By.ID, "username")
    )
)
```

Invisibility

```
wait.until(
    EC.invisibility_of_element_located(
        (By.ID, "loader")
    )
)
```

Alert

```
wait.until(
    EC.alert_is_present()
)
```

URL

```
wait.until(
    EC.url_contains("dashboard")
)
```

Text

```
wait.until(
    EC.text_to_be_present_in_element(
        (By.ID, "message"),
        "Success"
    )
)
```

5. Implicit Wait

```
driver.implicitly_wait(10)
```

Interview question

Implicit vs Explicit wait?

Implicit	Explicit
Global	Specific condition
Applies to element lookup	Applies to a particular wait
Simple	More powerful
Can cause complicated timing when mixed	Preferred for dynamic UI

For a modern framework, I generally prefer **explicit waits** for important synchronization.

6. Frames / IFrames ★★★★★

Your source contains many frame-switching variations, including frame by ID, name, index and WebElement, plus returning to the main document.

By ID

```
driver.switch_to.frame("payment_frame")
```

By WebElement

```
frame = driver.find_element(
    By.ID,
    "payment_frame"
)
```

```
driver.switch_to.frame(frame)
```

By index

```
driver.switch_to.frame(0)
```

Return to main document

```
driver.switch_to.default_content()
```

Parent frame

```
driver.switch_to.parent_frame()
```

Best approach

```
wait.until(
    EC.frame_to_be_available_and_switch_to_it(
        (By.ID, "payment_frame")
    )
)
```

7. JavaScript ★★☆☆

Your source includes JavaScript execution, scrolling, clicking, changing attributes, retrieving HTML and checking CSS properties.

```
driver.execute_script(
    "return document.title;"
)
```

Scroll

```
driver.execute_script(
    "arguments[0].scrollIntoView(true);",
    element
)
```

JS click

```
driver.execute_script(
    "arguments[0].click();",
    element
)
```

Get inner HTML

```
html = driver.execute_script(
    "return arguments[0].innerHTML;",
    element
)
```

Highlight element

```
driver.execute_script(
    "arguments[0].style.border='2px solid red';",
    element
)
```

Interview point

Don't use JavaScript click as your first solution.

Try:

Wait

↓

Scroll

↓

Normal click

↓

Actions click

↓

JavaScript click as last resort

8. Alerts ★★☆☆

The source includes accept, dismiss, get text and send keys for alerts.

```
alert = driver.switch_to.alert
```

```
print(alert.text)
```

```
alert.accept()
```

Dismiss

```
driver.switch_to.alert.dismiss()
```

Enter text

```
driver.switch_to.alert.send_keys(
    "Neeraj"
)
```

Wait for alert

```
wait.until(
    EC.alert_is_present()
)
```

```
driver.switch_to.alert.accept()
```

9. Windows / Tabs ★★☆☆

The source covers window handles, switching, closing and quitting.

```
current_window = driver.current_window_handle
```

```
windows = driver.window_handles
```

```
print(windows)
```

Switch:

```
driver.switch_to.window(
    windows[1]
)
```

Real-world pattern

```
main_window = driver.current_window_handle
```

```
# action that opens new tab
```

```
wait.until(
    lambda d: len(d.window_handles) > 1
)
```

```
for window in driver.window_handles:
```

```
    if window != main_window:
```

```
driver.switch_to.window(window)
```

```
print(driver.title)
```

```
driver.close()
```

```
break
```

```
driver.switch_to.window(main_window)
```

10. Cookies ★★☆☆

The source includes adding, deleting and retrieving cookies.

```
driver.add_cookie({  
    "name": "username",  
    "value": "Neeraj"  
})
```

Get:

```
cookies = driver.get_cookies()
```

```
print(cookies)
```

Specific cookie:

```
cookie = driver.get_cookie("username")
```

Delete:

```
driver.delete_cookie("username")
```

Delete all:

```
driver.delete_all_cookies()
```

11. Screenshots ★★☆☆

Very important for your automation framework.

```
driver.save_screenshot(  
    "failure.png"
```

```
)
```

Or:

```
element.screenshot(  
    "element.png"  
)
```

pytest failure screenshot

A common framework approach:

```
@pytest.hookimpl(hookwrapper=True)
```

```
def pytest_runtest_makereport(item, call):
```

```
    outcome = yield
```

```
    report = outcome.get_result()
```

```
if report.when == "call" and report.failed:
```

```
    driver = item.funcargs.get("driver")
```

```
    if driver:
```

```
        driver.save_screenshot(  
            "failure.png"  
        )
```

This is a **good 6-year-experience interview topic** because it demonstrates framework knowledge rather than only Selenium commands.

12. Keyboard & Mouse / Actions ★★☆☆

Your source categorizes these as advanced user interactions.

```
from selenium.webdriver.common.action_chains
```

```
import ActionChains
```

```
from selenium.webdriver.common.keys import Keys
```

```
actions = ActionChains(driver)
```

Hover

```
actions.move_to_element(  
    element  
) .perform()
```

Double click

```
actions.double_click(  
    element  
) .perform()
```

Right click

```
actions.context_click(  
    element  
) .perform()
```

Drag and drop

```
actions.drag_and_drop(  
    source,  
    target  
) .perform()
```

Keyboard

```
actions.send_keys(  
    Keys.ENTER  
) .perform()
```

Ctrl+A

```
actions.key_down(Keys.CONTROL)\  
    .send_keys("a")\  
    .key_up(Keys.CONTROL)\  
    .perform()
```

13. File Upload ★★☆☆

Your source shows uploading by sending the file path to an `<input type="file">`.

```
upload = driver.find_element(
    By.CSS_SELECTOR,
    "input[type='file']"
)
```

```
upload.send_keys(
    r"C:\test_data\sample.pdf"
)
```

Important

You normally **don't need to interact with the Windows file chooser**.

14. Browser Options

The source includes Chrome options such as maximized, headless, incognito, disabling extensions, GPU and sandbox settings.

Python:

```
from selenium import webdriver
```

```
options = webdriver.ChromeOptions()
```

```
options.add_argument("--start-maximized")
options.add_argument("--headless=new")
options.add_argument("--disable-extensions")
```

```
driver = webdriver.Chrome(
    options=options
)
```

Common CI option

```
options.add_argument("--headless=new")
```

Useful when running through Jenkins/GitLab CI.

15. Browser Logs

Your source also includes browser log retrieval for debugging.

In modern Selenium/Chrome setups, logging configuration varies by browser/version, but the concept is:

Browser

↓

Console errors

↓

Selenium/driver logging

↓

Test report

This is useful for investigating JavaScript failures, network problems, etc.

16. Timeouts

Your source separates WebDriver timeout commands into implicit wait, page-load timeout and script timeout.

Python:

```
driver.implicitly_wait(10)
```

```
driver.set_page_load_timeout(60)
```

```
driver.set_script_timeout(30)
```

17. Hidden Advertisement / Overlay Popup

★★★★★

This is the topic you asked about earlier.

First approach

try:

```
close_button = WebDriverWait(
    driver, 3
).until(
    EC.element_to_be_clickable(
        (By.CSS_SELECTOR, ".close-ad")
    )
)
```

```
close_button.click()
```

except TimeoutException:

```
    pass
```

Then:

```
wait.until(
    EC.element_to_be_clickable(
        (By.ID, "submit")
    )
).click()
```

If overlay disappears

```
wait.until(
    EC.invisibility_of_element_located(
        (By.CSS_SELECTOR, ".overlay")
    )
)
```

Last resort

```
driver.execute_script("""
```



```
document.querySelector(
    'advertisement-overlay'
)?.remove();
""")
```

18. Selenium Exceptions ★★★★★

For a 6-year interview, don't just memorize commands. Know **why failures happen**.

NoSuchElementException

Element cannot be found

Possible causes:

Wrong locator

Wrong page

Iframe

Element not loaded

TimeoutException

Expected condition didn't happen within timeout

ElementClickInterceptedException

Another element is blocking the click

Typical cause:

Advertisement

Modal

Overlay

Loader

Sticky header

ElementNotInteractableException

Element exists but cannot currently be interacted with.

StaleElementReferenceException

The element was found earlier, but the DOM changed and the old reference is no longer valid.

Typical solution:

```
element = wait.until(
    EC.element_to_be_clickable(locator)
)
```

```
element.click()
```

Re-locate the element rather than blindly reusing the old reference.

UnexpectedAlertPresentException

An unexpected JavaScript alert is blocking the operation.

19. Pytest Integration ★★★★★

Selenium alone isn't enough for your interview. You should know how Selenium fits into pytest.

Fixture

```
import pytest
```

```
from selenium import webdriver
```

```
@pytest.fixture
```

```
def driver():
```

```
    driver = webdriver.Chrome()
```

```
    driver.maximize_window()
```

```
    yield driver
```

```
    driver.quit()
```

Test:

```
def test_login(driver):
```

```
    driver.get(
        "https://example.com"
    )
```

```
    driver.find_element(
        By.ID,
        "username"
    ).send_keys("Neeraj")
```

20. Better Framework Structure

For your type of automation framework, I recommend remembering this structure:

```
automation_framework/
|
|— tests/
|   |— test_login.py
|   |— test_dashboard.py
|   |— test_users.py
|
|— pages/
|   |— login_page.py
|   |— dashboard_page.py
|   |— users_page.py
|
|— utilities/
|   |— waits.py
```

```

├── screenshot.py
├── popup_handler.py
└── common_methods.py

├── conftest.py

├── config/
│   └── config.ini
└── reports/

```

POM

class LoginPage:

```

USERNAME = (By.ID, "username")
PASSWORD = (By.ID, "password")
LOGIN = (By.ID, "login")

```

```

def __init__(self, driver):
    self.driver = driver

```

def login(self, username, password):

```

    self.driver.find_element(
        *self.USERNAME
    ).send_keys(username)

```

```

    self.driver.find_element(
        *self.PASSWORD
    ).send_keys(password)

```

```

    self.driver.find_element(
        *self.LOGIN
    ).click()

```

21. Your Interview Revision Priority

Don't spend equal time on all commands.

For a **6-year Python Selenium + pytest interview**, revise in this order:

● Must Know

1. Locators
2. XPath
3. CSS selectors
4. Explicit waits
5. Implicit vs Explicit wait
6. Frames/IFrames
7. Windows/Tabs
8. Alerts

9. Selenium exceptions
10. Actions
11. JavaScript
12. POM
13. Pytest fixtures
14. Parametrize
15. conftest.py
16. Screenshots on failure
17. Parallel execution / pytest-xdist
18. Selenium Grid
19. CI/CD
20. Handling flaky tests

● Important

21. Cookies
22. File upload/download
23. Browser options
24. Headless execution
25. Browser logs
26. Page Load Strategy
27. Shadow DOM
28. Dynamic elements
29. Stale elements
30. Retry mechanisms

● Secondary

31. Proxy
32. Remote WebDriver
33. Mobile Web/Appium
34. Advanced capabilities
35. Performance/logging

Your uploaded material contains a **large command reference**, but for interview preparation I would **not memorize all of it**. The high-value goal is to understand **when to use each category, why Selenium fails, and how you would build reusable utilities around it**.

In Selenium, the **Options class** is used to configure and customize browser behavior before launching the browser instance.

It helps you control things like:

- Headless mode
- Incognito/private mode
- Download settings
- Browser arguments
- Extensions
- Notifications
- Proxy
- Certificates
- Experimental settings

The Options class is different for each browser.

1. Chrome Options

Using the Chrome browser:

```
from selenium import webdriver
from selenium.webdriver.chrome.options
import Options
```

```
options = Options()
```

```
driver = webdriver.Chrome(options=options)
```

2. Why We Use Options Class

Without options:

```
driver = webdriver.Chrome()
```

With options:

```
options = Options()
```

```
options.add_argument("--start-maximized")
```

```
driver = webdriver.Chrome(options=options)
```

Now Chrome starts maximized.

3. Commonly Used Browser Options

A. Headless Mode

Runs browser without opening UI.

Useful in:

- CI/CD
- Jenkins
- GitLab pipeline
- Docker
- Fast execution

```
options = Options()
```

```
options.add_argument("--headless")
```

```
driver = webdriver.Chrome(options=options)
```

Modern Chrome:

```
options.add_argument("--headless=new")
```

B. Start Maximized

```
options.add_argument("--start-maximized")
```

C. Disable Notifications

Stops popup notifications.

```
options.add_argument("--disable-
notifications")
```

Useful for:

- Facebook
- Gmail
- Internal enterprise apps

D. Incognito Mode

```
options.add_argument("--incognito")
```

Useful when:

- No cache needed
- Clean session testing
- Cookie isolation

E. Disable Extensions

```
options.add_argument("--disable-extensions")
```

F. Ignore Certificate Errors

```
options.add_argument("--ignore-certificate-
errors")
```

Useful in:

- QA/UAT environments
 - Self-signed SSL sites
-

G. Disable Popup Blocking

```
options.add_argument("--disable-popup-blocking")
```

H. Open Specific User Profile

Very useful in automation frameworks.

```
options.add_argument(r"--user-data-dir=C:\Users\Admin\AppData\Local\Google\Chrome\User Data")
```

Use cases:

- Reuse logged-in session
- Avoid repeated login
- Persistent cookies

I. Remote Debugging Port

Useful for attaching Selenium to existing Chrome.

```
options.add_argument("--remote-debugging-port=9222")
```

4. Experimental Options

Used for advanced browser configuration.

Disable "Chrome is being controlled by automated software"

```
options.add_experimental_option("excludeSwitches", ["enable-automation"])
```

Disable Automation Banner

```
options.add_experimental_option("useAutomationExtension", False)
```

5. Download File Automatically

Very important in real-time automation frameworks.

```
prefs = {  
    "download.default_directory":  
r"D:\Downloads",  
    "download.prompt_for_download": False,  
    "download.directory_upgrade": True  
}
```

```
options.add_experimental_option("prefs",  
prefs)
```

Complete Example

```
from selenium import webdriver
```

```
from selenium.webdriver.chrome.options
```

```
import Options
```

```
options = Options()
```

```
prefs = {  
    "download.default_directory":  
r"D:\Downloads",  
    "download.prompt_for_download": False  
}
```

```
options.add_argument("--start-maximized")
```

```
options.add_argument("--disable-notifications")
```

```
options.add_argument("--incognito")
```

```
options.add_argument("--headless=new")
```

```
options.add_experimental_option("prefs",  
prefs)
```

```
driver = webdriver.Chrome(options=options)
```

```
driver.get("https://www.google.com")
```

6. Firefox Options

Firefox uses a different class.

```
from selenium.webdriver.firefox.options
```

```
import Options
```

```
options = Options()
```

```
options.add_argument("--headless")
```

```
driver = webdriver.Firefox(options=options)
```

7. Edge Options

```
from selenium.webdriver.edge.options import
Options
```

```
options = Options()
options.add_argument("--start-maximized")
```

```
driver = webdriver.Edge(options=options)
```

8. Real-Time Framework Usage

In advanced frameworks (like your Selenium + pytest frameworks), usually options are configured inside:

- conftest.py
- Driver factory
- Browser manager class
- Base test setup

Example:

```
@pytest.fixture
```

```
def setup():
```

```
    options = Options()
```

```
    if headless:
```

```
        options.add_argument("--headless=new")
```

```
    options.add_argument("--disable-
notifications")
```

```
    options.add_argument("--start-maximized")
```

```
    driver = webdriver.Chrome(options=options)
```

```
    yield driver
```

```
    driver.quit()
```

9. Interview Questions

Q1. What is the purpose of the Options class in Selenium?

The Options class is used to customize browser behavior before browser launch by passing browser arguments, preferences,

extensions, experimental settings, and capabilities.

Q2. Difference between add_argument() and add_experimental_option()

Method	Purpose
add_argument()	Pass browser command-line arguments
add_experimental_option()	Advanced Chrome internal configurations

10. Important Browser Arguments

Argument	Purpose
--headless	Run without UI
--incognito	Private mode
--start-maximized	Maximize window
--disable-notifications	Disable notifications
--disable-popup-blocking	Disable popups
--ignore-certificate-errors	Ignore SSL errors
--disable-gpu	Disable GPU rendering
--no-sandbox	Linux/Docker execution
--disable-dev-shm-usage	Avoid shared memory issue

11. Production-Level Chrome Setup

Common in CI/CD pipelines:

```
options = Options()
```

```
options.add_argument("--headless=new")
```

```
options.add_argument("--no-sandbox")
```

```
options.add_argument("--disable-dev-shm-
usage")
```

```
options.add_argument("--disable-gpu")
```

```
options.add_argument("--window-size=1920,1080")

driver = webdriver.Chrome(options=options)
Useful in:
```

- Jenkins
- Docker
- Linux server
- GitLab CI
- Kubernetes execution

The main difference between `add_argument()` and `add_experimental_option()` in Selenium is:

Method	Used For	Level
<code>add_argument()</code>	Passing browser command-line arguments	Browser startup arguments
<code>add_experimental_option()</code>	Setting internal Chrome preferences/configurations	Advanced Chrome settings

1. add_argument()

Used to send **Chrome command-line flags**.

Syntax:

```
options.add_argument("argument")
```

These arguments are the same flags you can pass when manually starting Chrome from terminal.

Example 1: Headless Mode

```
options.add_argument("--headless=new")
```

Meaning:

- Launch browser without UI.

Example 2: Incognito Mode

```
options.add_argument("--incognito")
```

Meaning:

- Open browser in private mode.

Example 3: Start Maximized

```
options.add_argument("--start-maximized")
```

Meaning:

- Start browser in maximized state.

Common Uses of add_argument()

Argument	Purpose
<code>--headless</code>	Run browser without UI
<code>--incognito</code>	Open private mode
<code>--disable-notifications</code>	Disable browser notifications
<code>--start-maximized</code>	Maximize browser
<code>--disable-gpu</code>	Disable GPU
<code>--no-sandbox</code>	Linux/Docker compatibility
<code>--window-size=1920,1080</code>	Set browser size

2. add_experimental_option()

Used for:

- Internal Chrome preferences
- Chrome behavior customization
- Advanced browser configuration

Syntax:

```
options.add_experimental_option(name, value)
```

Example 1: Disable Automation Banner

```
options.add_experimental_option("excludeSwitches", ["enable-automation"])
```

This hides:
Chrome is being controlled by automated software

Example 2: Download Settings

```
prefs = {  
    "download.default_directory":  
    r"D:\Downloads",  
    "download.prompt_for_download": False  
}
```

```
options.add_experimental_option("prefs",  
prefs)
```

- This:
- Changes download path
 - Disables download popup

Example 3: Disable Password Manager

```
prefs = {  
    "credentials_enable_service": False,  
    "profile.password_manager_enabled": False  
}
```

```
options.add_experimental_option("prefs",  
prefs)
```

Key Technical Difference

add_argument()

Works at:

- Browser startup command-line level

Equivalent to:

```
chrome.exe --headless --incognito
```

It passes startup flags directly to Chrome executable.

add_experimental_option()

Works at:

- Chrome internal configuration level

It modifies:

- Chrome preferences

- Automation settings
- Internal browser behaviors

Real-Time Analogy

Think of Chrome as a car.

add_argument()

Controls:

- How the car starts

Examples:

- Start in sports mode
- Start silently
- Start without display

add_experimental_option()

Controls:

- Internal settings of the car

Examples:

- Seat memory
- Navigation preferences
- Auto-lock settings

Framework-Level Example

```
from selenium import webdriver  
from selenium.webdriver.chrome.options  
import Options  
  
options = Options()  
  
# Browser startup arguments  
options.add_argument("--headless=new")  
options.add_argument("--start-maximized")  
  
# Internal Chrome settings  
prefs = {  
    "download.default_directory":  
    r"D:\Downloads",  
    "download.prompt_for_download": False  
}
```

```
options.add_experimental_option("prefs",  
prefs)
```

```
driver = webdriver.Chrome(options=options)
```

Interview Answer

What is the difference between `add_argument()` and `add_experimental_option()` in Selenium?

`add_argument()` is used to pass browser startup command-line arguments like headless mode, incognito mode, and window size.

`add_experimental_option()` is used to configure internal Chrome settings such as download preferences, automation control settings, and browser behavior customizations.

Important Note

`add_experimental_option()` is mostly:

- Chrome-specific
- Chromium-specific

But `add_argument()` works across:

- Chrome
- Edge
- Firefox (partially)
- Chromium browsers

Most Commonly Used Combination in Industry

```
options.add_argument("--headless=new")
```

```
options.add_argument("--disable-  
notifications")
```

```
prefs = {  
    "download.default_directory": r"D:\Reports"  
}
```

```
options.add_experimental_option("prefs",  
prefs)
```

This is very common in:

- Selenium frameworks
- CI/CD pipelines
- Enterprise automation
- Pytest frameworks
- Jenkins execution

In Selenium, **waits** are used to synchronize the automation script with the web application.

A senior automation engineer never relies on random: `time.sleep()`

because modern applications are dynamic:

- AJAX loading
- slow APIs
- animations
- delayed elements
- page rendering

Instead, Selenium provides proper wait mechanisms.

✔ **Types of Waits in Selenium**

There are mainly **4 wait strategies**

Wait Type	Purpose
Implicit Wait	global element wait
Explicit Wait	wait for specific condition
Fluent Wait	custom polling wait
Static Wait	fixed sleep

✔ **1. Implicit Wait**

Implicit wait tells Selenium:

“wait for a certain time while finding elements.”

Syntax

```
driver.implicitly_wait(10)
```

Meaning:

Selenium waits up to **10 seconds** before throwing exception.

Example

```
driver.implicitly_wait(10)
```



```
element = driver.find_element(By.ID, "username")  
element.send_keys("Admin")
```

How it works

Selenium checks repeatedly until:

✔ element found

or

✗ timeout reached

Scope

Applies globally for entire driver session.

Best for

Simple projects.

Limitation

Not ideal for:

- clickable state
- visibility
- alerts
- frames

✔ **2. Explicit Wait (Most Important)**

Explicit wait waits for a **specific condition**.

Senior engineers mostly use this.

Import

```
from selenium.webdriver.support.ui import  
WebDriverWait  
  
from selenium.webdriver.support import  
expected_conditions as EC
```

Syntax

```
WebDriverWait(driver, 10).until(condition)
```

Example – wait for visibility

```
element = WebDriverWait(driver, 10).until(  
    EC.visibility_of_element_located((By.ID,  
    "username"))  
)  
  
element.send_keys("Admin")
```

Example – wait for clickable

```
button = WebDriverWait(driver, 10).until(  
    EC.element_to_be_clickable((By.ID, "loginBtn"))  
)  
  
button.click()
```

Common conditions	
Condition	Use
presence_of_element_located	element exists
visibility_of_element_located	visible
element_to_be_clickable	clickable
alert_is_present	alert
frame_to_be_available_and_switch_to_it	frame

Condition	Use
number_of_windows_to_be	windows
invisibility_of_element_located	loader gone

Example for loader

```
WebDriverWait(driver, 20).until(
    EC.invisibility_of_element_located((By.ID, "loader"))
)
```

- Why preferred?**
- ✓ more stable
 - ✓ faster
 - ✓ professional framework approach

✓ **3. Fluent Wait**

Fluent wait allows:

- ✓ timeout
- ✓ polling frequency
- ✓ ignored exceptions

Used in advanced frameworks.

Import

from selenium.webdriver.support.wait import
WebDriverWait

Python uses WebDriverWait with polling.

Example

```
wait = WebDriverWait(
    driver,
    timeout=20,
    poll_frequency=2,
    ignored_exceptions=[Exception]
)
element = wait.until(
    EC.visibility_of_element_located((By.ID,
"username"))
)
```

- Meaning**
- total wait → 20 sec
 - check every → 2 sec
 - ignore exceptions

When useful

For unstable dynamic pages.

- ✓ **4. Static Wait (time.sleep)**
- Forces execution to stop.
- When time.sleep() is Actually Acceptable:**
- **Debugging** during script development
 - **Rate limiting** (API calls with hard limits)
 - **Simulating human behavior** in demo scripts
 - **External hardware/device waiting** (no programmatic feedback)

Rule of thumb: If you can detect when the condition is met, use explicit waits. Only use time.sleep() when you have **no observable condition** to wait for.

Example

```
import time
time.sleep(5)
```

Problem

Always waits full time even if element appears early.
Bad for frameworks.

Example problem

```
time.sleep(10)
```

Even if page loads in 2 sec, script waits 10 sec.

Use only for

Temporary debugging.

✓ **Custom Wait Using Lambda**

Senior engineers often create custom conditions.

Example

```
Wait until text changes
WebDriverWait(driver, 10).until(
    lambda d: d.find_element(By.ID, "status").text ==
"Completed"
)
```

Example attribute wait

```
WebDriverWait(driver, 10).until(
    lambda d: d.find_element(By.ID,
"save").get_attribute("disabled") is None
)
```

✓ **Reusable Wait Utility Class (Framework Style)**

Professional framework example.

```
class WaitHelper:

    def __init__(self, driver):
        self.driver = driver

    def wait_visible(self, locator, timeout=10):
        return WebDriverWait(self.driver, timeout).until(
            EC.visibility_of_element_located(locator)
        )

    def wait_clickable(self, locator, timeout=10):
        return WebDriverWait(self.driver, timeout).until(
            EC.element_to_be_clickable(locator)
        )

    def wait_invisible(self, locator, timeout=10):
        return WebDriverWait(self.driver, timeout).until(
            EC.invisibility_of_element_located(locator)
        )
```

Usage

```
wait = WaitHelper(driver)

wait.wait_visible((By.ID,
"username")).send_keys("Admin")
wait.wait_clickable((By.ID, "login")).click()
```

Real Project Example

```
Loader handling.

WebDriverWait(driver, 20).until(

    EC.invisibility_of_element_located((By.CLASS_NAME,
"loading"))
)

driver.find_element(By.ID, "submit").click()
```

Without wait

```
Could fail with:
ElementClickInterceptedException
```

Mixing Waits (Avoid)

```
Avoid mixing:
driver.implicitly_wait(10)
WebDriverWait(driver, 20)

May cause unpredictable timing.
```

Best practice

- ✓ Prefer explicit waits only.

Common Exceptions Prevented by Waits

Exception	Prevented by
NoSuchElementException	visibility wait
ElementNotInteractableException	clickable wait
TimeoutException	better synchronization
StaleElementReferenceException	refreshed wait

Comparison Table

Wait	Best For	Recommended
Implicit	simple	limited
Explicit	specific element	highly recommended
Fluent	advanced	recommended
Static	debugging	avoid

Senior Best Practices

- ✓ Use explicit waits in page objects
- ✓ Build reusable wait methods
- ✓ Wait for state, not just presence
- ✓ Never overuse sleep
- ✓ Handle loaders and AJAX properly
- ✓ Keep timeout configurable

Interview Answer

Selenium provides four wait mechanisms: implicit wait for global element searching, explicit wait for condition-based synchronization, fluent wait for custom polling intervals, and static waits using sleep. In enterprise automation frameworks, explicit waits with reusable helper methods are preferred because they improve script stability and reduce flaky failures.

Most Common Waits in Real Projects

```
Most used:
EC.visibility_of_element_located()
EC.element_to_be_clickable()
EC.invisibility_of_element_located()
EC.alert_is_present()
```

EC.frame_to_be_available_and_switch_to_it()

If you'd like, I can explain **how to handle StaleElementReferenceException using waits**, because that is one of the most common advanced Selenium issues in large frameworks.

1. Use Fluent Wait when:

- Elements load with highly variable timing
- Dealing with unstable/legacy web applications
- Need fine-grained control over polling frequency
- Multiple exception types need to be ignored

2. Avoid Fluent Wait when:

- Standard explicit wait is sufficient
- Performance is critical (polling adds overhead)
- Condition is simple and predictable

3. Optimization Tips:

- Set poll_frequency lower (0.1-0.3s) for fast-reacting conditions
- Set poll_frequency higher (1-2s) for slow backend processes
- Always include timeout limits to prevent infinite hangs

Selenium Expected Conditions (EC) with Related Exceptions

Import Required

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import
expected_conditions as EC
```

Why Expected Conditions Are Used

Expected Conditions help prevent Selenium exceptions by waiting dynamically until a condition becomes true.

Without waits, Selenium may fail because:

- element not loaded
- element hidden
- page not refreshed
- frame not available
- loader still visible
- alert not opened

1. title_is()

Purpose

Wait until page title exactly matches.

Syntax

```
EC.title_is("Dashboard")
```

Prevents

AssertionError

Use Case

After login or navigation.

2. title_contains()

Purpose

Wait until title contains partial text.

Syntax

```
EC.title_contains("Dashboard")
```

Prevents

AssertionError

Use Case

Dynamic page titles.

3. url_to_be()

Purpose

Wait until URL exactly matches.

Syntax

```
EC.url_to_be("https://example.com/home")
```

Prevents

AssertionError

Use Case

Redirect validation.

4. url_contains()

Purpose

Wait until URL contains text.

Syntax

```
EC.url_contains("dashboard")
```

Prevents

AssertionError

5. url_matches()

Purpose

Wait until URL matches regex.

Syntax

```
EC.url_matches(".*dashboard.*")
```

Prevents

AssertionError

6. url_changes()

Purpose

Wait until URL changes.

Syntax

```
EC.url_changes(old_url)
```

Prevents

TimeoutException

Use Case

After clicking navigation button.

7. presence_of_element_located()

Purpose

Wait until element exists in DOM.

Syntax

```
EC.presence_of_element_located((By.ID, "username"))
```

Prevents

NoSuchElementException

Important

Element may still be hidden.

8. visibility_of_element_located()

Purpose

Wait until element becomes visible.

Syntax

```
EC.visibility_of_element_located((By.ID, "username"))
```

Prevents

ElementNotVisibleException

ElementNotInteractableException

NoSuchElementException

Most Commonly Used

9. visibility_of()

Purpose

Wait until existing WebElement becomes visible.

Syntax

```
EC.visibility_of(element)
```

Prevents

ElementNotVisibleException

ElementNotInteractableException

10. presence_of_all_elements_located()

Purpose

Wait until multiple elements exist.

Syntax

EC.presence_of_all_elements_located((By.TAG_NAME, "a"))

Prevents

NoSuchElementException

11. visibility_of_any_elements_located()

Purpose

Wait until at least one element becomes visible.

Syntax

EC.visibility_of_any_elements_located((By.CLASS_NAME, "item"))

Prevents

ElementNotVisibleException

12. visibility_of_all_elements_located()

Purpose

Wait until all elements become visible.

Syntax

EC.visibility_of_all_elements_located((By.CLASS_NAME, "item"))

Prevents

ElementNotVisibleException

ElementNotInteractableException

13. element_to_be_clickable()

Purpose

Wait until element becomes clickable.

Syntax

EC.element_to_be_clickable((By.ID, "login"))

Prevents

ElementClickInterceptedException

ElementNotInteractableException

ElementNotVisibleException

Most Commonly Used

14. invisibility_of_element_located()

Purpose

Wait until element disappears.

Syntax

EC.invisibility_of_element_located((By.ID, "loader"))

Prevents

ElementClickInterceptedException

Use Case

Loader/spinner handling.

15. invisibility_of_element()

Purpose

Wait until existing element becomes invisible.

Syntax

EC.invisibility_of_element(element)

Prevents

ElementClickInterceptedException

16. text_to_be_present_in_element()

Purpose

Wait until text appears inside element.

Syntax

EC.text_to_be_present_in_element(
 (By.ID, "msg"),
 "Success"
)

Prevents

AssertionError

Use Case

Success/error messages.

17. text_to_be_present_in_element_value()

Purpose

Wait until text appears inside input value.

Syntax

EC.text_to_be_present_in_element_value(
 (By.ID, "username"),
 "Admin"
)

Prevents

AssertionError

18. text_to_be_present_in_element_attribute()

Purpose

Wait until attribute contains expected text.

Syntax

EC.text_to_be_present_in_element_attribute(
 (By.ID, "field"),
 "value",
 "Admin"
)

Prevents

AssertionError

19. frame_to_be_available_and_switch_to_it()

Purpose

Wait until iframe becomes available and switch automatically.

Syntax

EC.frame_to_be_available_and_switch_to_it(
 (By.ID, "frame1")
)

Prevents

NoSuchFrameException

Most Useful for iframe handling.

20. alert_is_present()

Purpose

Wait until alert appears.

Syntax

```
EC.alert_is_present()
```

Prevents

NoAlertPresentException

21. number_of_windows_to_be()

Purpose

Wait until expected number of tabs/windows open.

Syntax

```
EC.number_of_windows_to_be(2)
```

Prevents

NoSuchWindowException

22. new_window_is_opened()

Purpose

Wait until new window opens.

Syntax

```
EC.new_window_is_opened(current_handles)
```

Prevents

NoSuchWindowException

23. staleness_of()

Purpose

Wait until old element becomes stale.

Syntax

```
EC.staleness_of(element)
```

Prevents

StaleElementReferenceException

Most Important for:

- page refresh
 - AJAX DOM reload
 - dynamic applications
-

24. element_to_be_selected()

Purpose

Wait until checkbox/radio button becomes selected.

Syntax

```
EC.element_to_be_selected(element)
```

Prevents

AssertionError

25. element_located_to_be_selected()

Purpose

Wait until located element becomes selected.

Syntax

```
EC.element_located_to_be_selected((By.ID, "check"))
```

Prevents

AssertionError

26. element_selection_state_to_be()

Purpose

Wait until selection state matches expected state.

Syntax

```
EC.element_selection_state_to_be(element, True)
```

Prevents

AssertionError

27. element_located_selection_state_to_be()

Purpose

Wait until located element state matches expected state.

Syntax

```
EC.element_located_selection_state_to_be(  
    (By.ID, "check"),  
    True  
)
```

Prevents

AssertionError

Real-Time Example

```
wait = WebDriverWait(driver, 10)
```

```
username = wait.until(  
    EC.visibility_of_element_located(  
        (By.ID, "username")  
    )  
)
```

```
login_btn = wait.until(  
    EC.element_to_be_clickable(  
        (By.ID, "login")  
    )  
)
```

```
login_btn.click()
```

```
wait.until(  
    EC.title_contains("Dashboard")  
)
```

Most Important Expected Conditions in Real Projects

```
EC.visibility_of_element_located()  
EC.element_to_be_clickable()  
EC.presence_of_element_located()  
EC.invisibility_of_element_located()  
EC.frame_to_be_available_and_switch_to_it()  
EC.alert_is_present()  
EC.number_of_windows_to_be()  
EC.staleness_of()
```

Important Note

Expected Conditions do not completely eliminate exceptions.

They reduce synchronization-related failures by waiting properly before interaction.

Interview Summary

Expected Conditions are Selenium predefined wait conditions used with WebDriverWait to synchronize automation scripts with web application behavior. They help prevent common Selenium exceptions like NoSuchElementException, StaleElementReferenceException, ElementNotInteractableException, NoAlertPresentException, and NoSuchFrameException by dynamically waiting until elements, alerts, frames, URLs, or windows are ready for interaction.

Actions class use in java selenium but in python selenium it is ActionChains class
In **Python Selenium**, the **ActionChains** class is used to perform advanced user interactions that cannot be handled with normal click() or send_keys().

It is available in Selenium as:
from selenium.webdriver.common.action_chains
import ActionChains

✔ Why we use ActionChains

We use ActionChains for:

- Mouse hover
- Right click
- Double click
- Drag and drop
- Click and hold
- Keyboard combinations
- Move to element
- Complex UI interactions

✔ Import syntax

from selenium.webdriver.common.action_chains
import ActionChains

Create object:

actions = ActionChains(driver)

✔ General syntax

actions.method_name(element).perform()

Example:

actions.click(element).perform()

✔ How it works internally

Flow:

Create action

↓

Queue actions

↓

perform()

↓

Browser executes

Without perform(), action will not execute.

✔ Example — Mouse Hover

Used for menus.

menu = driver.find_element(By.ID, "menu")

actions = ActionChains(driver)
actions.move_to_element(menu).perform()

✔ Example — Double Click

button = driver.find_element(By.ID, "btn")

actions.double_click(button).perform()

✔ Example — Right Click

element = driver.find_element(By.ID, "box")

actions.context_click(element).perform()

✔ Example — Drag and Drop

source = driver.find_element(By.ID, "source")

target = driver.find_element(By.ID, "target")

actions.drag_and_drop(source, target).perform()

✔ Example — Click and Hold

element = driver.find_element(By.ID, "slider")

actions.click_and_hold(element).perform()

✔ Release mouse

actions.release().perform()

✔ Example — Keyboard actions

from selenium.webdriver.common.keys import Keys

actions.key_down(Keys.CONTROL).send_keys("a").key_up(Keys.CONTROL).perform()

Select all text.

✔ Example — Multiple chained actions

actions.move_to_element(menu).click(submenu).perform()

Meaning:

hover first

then click submenu

✔ Important methods

Method	Purpose
click()	Single click
double_click()	Double click
context_click()	Right click

Method	Purpose
<code>move_to_element()</code>	Hover
<code>drag_and_drop()</code>	Drag item
<code>click_and_hold()</code>	Hold mouse
<code>release()</code>	Release mouse
<code>send_keys()</code>	Keyboard input

✓ **move_by_offset()**

Move mouse by pixels.

```
actions.move_by_offset(100, 50).perform()
```

Moves:

100 px right

50 px down

✓ **Scroll example**

Sometimes useful:

```
actions.scroll_to_element(element).perform()
```

(Available in newer Selenium versions)

✓ **Real framework usage**

Example in automation:

```
def hover_and_click(driver, menu, submenu):
```

```
    actions = ActionChains(driver)
```

```
    actions.move_to_element(menu).click(submenu).perform()
```

Reusable helper.

✓ **Difference from normal click**

Normal click

```
element.click()
```

Simple click only.

ActionChains click

```
actions.click(element).perform()
```

More stable for complex UI.

✓ **Common issue**

Forgetting:

```
.perform()
```

Wrong:

```
actions.move_to_element(element)
```

Nothing happens.

Correct:

```
actions.move_to_element(element).perform()
```

✓ **Interview explanation**

“ActionChains in Selenium is used for advanced mouse and keyboard interactions such as hover, drag-and-drop, double-click, and composite UI actions that cannot be achieved reliably with standard WebElement methods.”

✓ **Common interview example**

Hover menu:

```
from selenium.webdriver.common.action_chains
import ActionChains
```

```
menu = driver.find_element(By.XPATH,
"//span[text()='Products']")
```

```
ActionChains(driver).move_to_element(menu).perform()
```

✓ **Chaining multiple actions**

```
actions.click_and_hold(source)\
    .move_to_element(target)\
    .release()\
    .perform()
```

✓ **Best practice**

Create once:

```
actions = ActionChains(driver)
```

Reuse in page class.

✓ **Time complexity**

ActionChains itself does not improve performance.

It improves:

✓ interaction reliability

✓ UI handling

✓ real-user simulation

✓ **In Page Object Model**

```
class HomePage:
```

```
    def hover_menu(self):
```

```
        ActionChains(self.driver).move_to_element(self.menu).perform()
```

Clean framework design.

If you'd like, I can explain the difference between

ActionChains and JavaScript executor click

because both are often used when normal Selenium clicks fail.

Selenium Special Keys (Keys) in Python

In Selenium, the Keys class is used to simulate **keyboard actions** such as pressing Enter, Tab, Arrow keys, Function keys, Control keys, etc.

It is imported from:

```
from selenium.webdriver.common.keys import Keys
```

Example:

```
element.send_keys("Hello", Keys.ENTER)
```

This types "Hello" and presses the **Enter** key.

Commonly Used Special Keys

1. Modifier Keys

These keys are used with other keys.

Key	Purpose	Example
Keys.ALT	Alt key	Alt + Tab
Keys.CONTROL	Ctrl key	Ctrl + A
Keys.SHIFT	Shift key	Shift + letter
Keys.COMMAND	Command key (Mac)	Command + C
Keys.META	Meta/Windows key	Windows key

Example:

```
element.send_keys(Keys.CONTROL, 'a') # Select all text
```

Arrow / Navigation Keys

Key	Purpose
Keys.ARROW_DOWN	Move cursor/selection down
Keys.ARROW_UP	Move up
Keys.ARROW_LEFT	Move left
Keys.ARROW_RIGHT	Move right
Keys.DOWN	Same as ARROW_DOWN
Keys.UP	Same as ARROW_UP
Keys.LEFT	Same as ARROW_LEFT
Keys.RIGHT	Same as ARROW_RIGHT

Example:

```
dropdown.send_keys(Keys.ARROW_DOWN)
dropdown.send_keys(Keys.ENTER)
```

Used in dropdown selection.

Editing Keys

Key	Purpose
Keys.BACKSPACE / Keys.BACK_SPACE	Delete previous character
Keys.DELETE	Delete next character

Key	Purpose
Keys.CLEAR	Clear field
Keys.INSERT	Insert mode
Keys.CANCEL	Cancel operation

Example:

```
textbox.send_keys(Keys.CONTROL, 'a')
textbox.send_keys(Keys.DELETE)
```

Action Keys

Key	Purpose
Keys.ENTER	Press Enter
Keys.TAB	Move to next field
Keys.ESCAPE	Escape/close popup
Keys.SPACE	Press spacebar

Example:

```
search.send_keys("Python", Keys.ENTER)
```

Function Keys

Key	Purpose
Keys.F1	Help
Keys.F2	Rename/edit
Keys.F5	Refresh page
Keys.F11	Full screen
Keys.F12	Developer tools

Example:

```
driver.find_element(By.TAG_NAME, "body").send_keys(Keys.F5)
```

Refreshes page.

Numeric Keypad Keys

Key	Purpose
Keys.NUMPAD0 to Keys.NUMPAD9	Numeric keypad numbers
Keys.ADD	+
Keys.SUBTRACT	-
Keys.MULTIPLY	*
Keys.DIVIDE	/
Keys.DECIMAL	Decimal point

Example:

```
element.send_keys(Keys.NUMPAD1)
```

Cursor Movement Keys

Key	Purpose
Keys.HOME	Go to start
Keys.END	Go to end
Keys.PAGE_UP	Scroll up
Keys.PAGE_DOWN	Scroll down
Example:	
body.send_keys(Keys.END)	
Scroll to bottom.	

Left-Side Specific Keys

Key	Purpose
Keys.LEFT_ALT	Left Alt key
Keys.LEFT_CONTROL	Left Ctrl key
Keys.LEFT_SHIFT	Left Shift key
Mostly used in advanced keyboard automation.	

Other Keys

Key	Purpose
Keys.EQUALS =	
Keys.NULL	Null/no key
Keys.HELP	Help key

Real-Time Examples

Press Enter

```
from selenium.webdriver.common.keys import Keys

driver.find_element(By.ID,
"search").send_keys("Laptop", Keys.ENTER)
```

Select All and Delete

```
textbox.send_keys(Keys.CONTROL, 'a')
textbox.send_keys(Keys.DELETE)
```

Handle Dropdown with Keyboard

```
dropdown.click()
dropdown.send_keys(Keys.ARROW_DOWN)
dropdown.send_keys(Keys.ARROW_DOWN)
dropdown.send_keys(Keys.ENTER)
```

Scroll Down Using Keyboard

```
body = driver.find_element(By.TAG_NAME, "body")
body.send_keys(Keys.END)
```

Important Notes

1. Keys works with send_keys()

```
element.send_keys(Keys.ENTER)
```

2. Multiple Keys Together

```
element.send_keys(Keys.CONTROL, 'c')
```

3. Import Required

```
from selenium.webdriver.common.keys import Keys
```

Most Frequently Used Keys in Automation

Frequently Used

- Keys.ENTER
- Keys.TAB
- Keys.CONTROL
- Keys.DELETE
- Keys.BACKSPACE
- Keys.ARROW_DOWN
- Keys.ARROW_UP
- Keys.ESCAPE
- Keys.HOME
- Keys.END

Dropdown selection

In Selenium, **dropdowns can appear in different forms**, and each type is handled differently.

There are mainly **3 types of dropdowns**:

1. **Standard HTML dropdown (<select>)**
2. **Custom dropdown (div, ul, li)**
3. **Auto-suggestion / searchable dropdown**

✅ 1. Standard HTML Dropdown (<select> tag)

This uses the Selenium **Select class**

HTML Example

```
<select id="country">
  <option value="1">India</option>
  <option value="2">USA</option>
  <option value="3">Canada</option>
</select>
```

Python Script

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
```

```
dropdown_element = driver.find_element(By.ID,
"country")
dropdown = Select(dropdown_element)
```

```
dropdown.select_by_visible_text("India")
```

Other methods

Select by index

```
dropdown.select_by_index(1)
```

Select by value

```
dropdown.select_by_value("2")
```

✅ 2. Custom Dropdown (div, span, li)

Some modern websites do not use <select>

Example:

```
<div id="countryDropdown">Select Country</div>
<ul>
  <li>India</li>
  <li>USA</li>
  <li>Canada</li>
</ul>
```

Select class will NOT work here.

Python Script

```
driver.find_element(By.ID, "countryDropdown").click()
driver.find_element(
```

```
By.XPATH,
  "//li[text()='India']"
).click()
```

Dynamic reusable function

```
def select_custom_dropdown(driver,
dropdown_locator, option_text):
    driver.find_element(*dropdown_locator).click()
```

```
    option = driver.find_element(
        By.XPATH,
        f"//li[text()=' {option_text}']"
    )
    option.click()
```

Usage:

```
select_custom_dropdown(driver, (By.ID,
"countryDropdown"), "India")
```

✅ 3. Auto-Suggestion Dropdown

These dropdowns show options while typing.

Example:

Google search, city search.

HTML Example

```
<input type="text" id="city">
<ul>
  <li>Delhi</li>
  <li>Mumbai</li>
</ul>
```

Python Script

```
search_box = driver.find_element(By.ID, "city")
search_box.send_keys("Del")
```

```
options = driver.find_elements(By.XPATH, "//ul/li")
```

for option in options:

```
    if option.text == "Delhi":
        option.click()
        break
```

✅ 4. Multi-select Dropdown

HTML:

```
<select multiple id="skills">
```

Python Script

```
skills = Select(driver.find_element(By.ID, "skills"))
```

```
skills.select_by_visible_text("Python")
skills.select_by_visible_text("Java")
```

Deselect

```
skills.deselect_by_visible_text("Python")
skills.deselect_all()
```

✔ **5. Hidden Dropdown (Need JavaScript)**

Sometimes dropdown is hidden.

Python Script

```
element = driver.find_element(By.ID, "country")

driver.execute_script(
    "arguments[0].value='2';", element
)
```

✔ **6. Bootstrap Dropdown**

Example:

```
<button class="dropdown-toggle">Select</button>
<div class="dropdown-menu">
    <a>India</a>
</div>
```

Python Script

```
driver.find_element(By.CLASS_NAME, "dropdown-
toggle").click()

driver.find_element(
    By.XPATH,
    "//a[text()='India']"
).click()
```

✔ **Universal Dropdown Handler Example**

```
You can create one utility method.
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select

def select_dropdown(driver, locator, value,
dropdown_type="select"):
    element = driver.find_element(*locator)

    if dropdown_type == "select":
        Select(element).select_by_visible_text(value)

    elif dropdown_type == "custom":
```

```
element.click()
driver.find_element(
    By.XPATH,
    f"//*[text()='{{value}}']"
).click()
```

```
Usage:
select_dropdown(driver, (By.ID, "country"), "India",
"select")
select_dropdown(driver, (By.ID, "cityDrop"), "Delhi",
"custom")
```

✔ **Best Practice with WebDriverWait**

```
Use waits before selecting.
from selenium.webdriver.support.ui import
WebDriverWait
from selenium.webdriver.support import
expected_conditions as EC

dropdown = WebDriverWait(driver, 10).until(
    EC.element_to_be_clickable((By.ID, "country"))
)

Select(dropdown).select_by_visible_text("India")
```

✔ **Summary Table**

Dropdown Type Handling Method	
<select>	Select class
div/li	click + xpath
autosuggestion	send_keys + loop
multi-select	Select class
hidden	JavaScript
bootstrap	click menu + option

✔ **Interview Answer**

In Selenium, dropdown handling depends on the dropdown type. Standard HTML dropdowns use the Select class, custom dropdowns require click and option selection, and auto-suggestion dropdowns require typing followed by iterating through matching options.

If you'd like, I can help you create a **framework-level reusable dropdown utility method** that handles all dropdown types in your Selenium pytest project.

Alert handling

Handling **alerts and pop-ups in Selenium** is a very common real-world requirement in enterprise automation.

A senior automation engineer usually classifies pop-ups into different categories because **not every popup is handled the same way**.

✔ Types of Alerts / Pop-ups in Selenium

There are mainly **4 categories**

Type	Example	Handling
JavaScript Alert	alert()	switch_to.alert
Confirmation Alert	confirm()	accept / dismiss
Prompt Alert	prompt()	send_keys
HTML Popup / Modal	Bootstrap modal	normal element handling
Browser Notification Popup	Chrome notification	browser options
File Upload Popup	OS popup	send_keys / external tools

✔ 1. JavaScript Alert

Simple alert with OK button.

Example JS

```
alert("Record saved successfully")
```

Python Script

```
alert = driver.switch_to.alert
print(alert.text)
alert.accept()
```

Explanation

Switch to alert

```
alert = driver.switch_to.alert
```

Read text

```
message = alert.text
```

Click OK

```
alert.accept()
```

✔ 2. Confirmation Alert

Has:

- OK
- Cancel

Example:

```
confirm("Are you sure?")
```

Accept

```
alert = driver.switch_to.alert
alert.accept()
```

Cancel

```
alert = driver.switch_to.alert
alert.dismiss()
```

✔ 3. Prompt Alert

Prompt accepts text input.

Example:

```
prompt("Enter your name")
```

Python Script

```
alert = driver.switch_to.alert
alert.send_keys("Neeraj")
alert.accept()
```

Explanation

Enter text

```
alert.send_keys("Neeraj")
```

Accept

```
alert.accept()
```

✔ 4. Check if Alert Exists

Use explicit wait.

```
from selenium.webdriver.support.ui import
WebDriverWait
from selenium.webdriver.support import
expected_conditions as EC
```

```
alert = WebDriverWait(driver, 10).until(
    EC.alert_is_present()
)
```

```
print(alert.text)
alert.accept()
```

Why senior engineers prefer this

Because direct:

```
driver.switch_to.alert
```

may fail with:

NoAlertPresentException

✔ 5. Reusable Alert Handler (Framework Style)

Reusable utility.

```
def handle_alert(driver, action="accept", text=None):
    try:
        alert = WebDriverWait(driver, 10).until(
            EC.alert_is_present()
        )

        message = alert.text

        if text:
            alert.send_keys(text)

        if action.lower() == "accept":
            alert.accept()
        elif action.lower() == "dismiss":
            alert.dismiss()

        return message

    except Exception as e:
        print(f"Alert not found: {e}")
        return None
```

Usage

```
msg = handle_alert(driver)
print(msg)
Prompt example:
handle_alert(driver, action="accept", text="Admin")
```

✔ 6. HTML Popup / Modal Dialog

Some applications use HTML modal popup.

Example:

Bootstrap popup.

HTML

```
<div class="modal">
  <button>Close</button>
</div>
```

Handle like normal element

```
popup = driver.find_element(By.CLASS_NAME,
"modal")
close_btn = popup.find_element(By.XPATH,
"./button[text()='Close']")
```

```
close_btn.click()
```

Senior approach with wait

```
popup = WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.CLASS_NAME,
"modal"))
)

popup.find_element(By.XPATH,
"./button[text()='Close']").click()
```

✔ 7. Browser Notification Popup

Chrome notification:

“Allow notifications?”

Selenium cannot directly click browser notification.

Best practice: disable before launch.

Chrome Options

```
from selenium import webdriver
```

```
options = webdriver.ChromeOptions()
options.add_argument("--disable-notifications")
```

```
driver = webdriver.Chrome(options=options)
```

Senior enterprise version

```
prefs = {

"profile.default_content_setting_values.notifications"
: 2
}
```

```
options = webdriver.ChromeOptions()
options.add_experimental_option("prefs", prefs)
```

```
driver = webdriver.Chrome(options=options)
```

✔ 8. File Upload Popup

System popup cannot be handled directly.

Best practice:

Use send_keys().

```
driver.find_element(By.ID,
"upload").send_keys("C:/files/test.pdf")
```

No need to interact with OS popup.

✔ 9. Unexpected Alert Handling

Sometimes alerts appear unexpectedly.

```
from selenium.common.exceptions import
UnexpectedAlertPresentException
try:
    driver.find_element(By.ID, "save").click()
except UnexpectedAlertPresentException:
    alert = driver.switch_to.alert
    print(alert.text)
    alert.accept()
```

✔ 10. Verify Alert Text

Validation step.

```
alert = WebDriverWait(driver, 10).until(
    EC.alert_is_present()
)
assert alert.text == "Record saved successfully"
alert.accept()
```

✔ Advanced Framework Utility Class

Senior reusable class.

```
class AlertHandler:

    def __init__(self, driver):
        self.driver = driver

    def wait_for_alert(self, timeout=10):
        return WebDriverWait(self.driver, timeout).until(
            EC.alert_is_present()
        )

    def accept_alert(self):
        self.wait_for_alert().accept()

    def dismiss_alert(self):
        self.wait_for_alert().dismiss()

    def get_alert_text(self):
        return self.wait_for_alert().text

    def enter_text(self, text):
        self.wait_for_alert().send_keys(text)
```

Usage

```
alert = AlertHandler(driver)
print(alert.get_alert_text())
alert.accept_alert()
```

✔ Common Exceptions

Exception	Reason
NoAlertPresentException	No alert found
UnexpectedAlertPresentException	Alert interrupted execution
TimeoutException	Alert did not appear

✔ Best Practices (Senior Level)

- ✔ Always use explicit wait
- ✔ Create reusable utility
- ✔ Validate alert message
- ✔ Handle unexpected alerts
- ✔ Disable browser notifications before launch
- ✔ Distinguish JS alert vs HTML popup

✔ Real Project Example

```
Save operation popup.
driver.find_element(By.ID, "saveBtn").click()

alert = WebDriverWait(driver, 10).until(
    EC.alert_is_present()
)
assert "saved" in alert.text.lower()

alert.accept()
```

✔ Interview Answer

In Selenium, JavaScript alerts are handled using `driver.switch_to.alert`, where `accept()`, `dismiss()`, and `send_keys()` are used based on alert type. For HTML popups, standard element locators are used. In enterprise frameworks, alert handling is usually wrapped inside reusable methods with explicit waits to improve reliability and maintainability.

If you'd like, I can explain **how to handle hidden advertisement popups and overlay popups that block clicking elements in Selenium**, which is a common real-project issue.

Handling **frames and iframes in Selenium** is another important real-world automation topic because many enterprise applications load:

- forms inside iframes
 - payment gateways
 - editors
 - dashboards
 - embedded documents
-

A senior automation engineer handles frames carefully because Selenium **cannot directly access elements inside a frame until the driver switches into it.**

✔ What is a Frame / iFrame

A frame is an HTML document embedded inside another page.

Example HTML

```
<iframe id="paymentFrame">
  <html>
    <input id="cardNumber">
  </html>
</iframe>
```

Without switching into the frame, Selenium cannot find:

```
driver.find_element(By.ID, "cardNumber")
```

It throws:

NoSuchElementException

✔ Why switching is required

Selenium initially works only in:
default content

To access frame elements:

```
driver.switch_to.frame()
```

✔ Types of frame switching

Selenium supports:

Method	Usage
by index	frame(0)
by name/id	frame("frameName")
by WebElement	frame(element)

✔ 1. Switch by Index

Use when only one frame exists.

```
driver.switch_to.frame(0)
```

Then interact:

```
driver.find_element(By.ID, "username").send_keys("Admin")
```

Return:

```
driver.switch_to.default_content()
```



2. Switch by Name or ID

Best if frame has unique ID.

HTML

```
<iframe id="loginFrame"></iframe>
```

Script

```
driver.switch_to.frame("loginFrame")
```

or

```
driver.switch_to.frame("frameName")
```



3. Switch by WebElement (Recommended)

Most reliable in frameworks.

```
frame = driver.find_element(By.ID, "loginFrame")
```

```
driver.switch_to.frame(frame)
```

Then:

```
driver.find_element(By.ID, "username").send_keys("Admin")
```



4. Switch back to Main Page

Very important.

```
driver.switch_to.default_content()
```

Returns to parent page.



5. Switch to Parent Frame

When nested frames exist.

```
driver.switch_to.parent_frame()
```

Moves one level up.



Nested Frames Example

HTML structure

```
<iframe id="outer">
```

```
    <iframe id="inner">
```

```
        <input id="txtName">
```

```
    </iframe>
```

```
</iframe>
```

Script

```
driver.switch_to.frame("outer")
```

```
driver.switch_to.frame("inner")
```

```
driver.find_element(By.ID, "txtName").send_keys("Neeraj")
```

Return one level:

```
driver.switch_to.parent_frame()
```

Return root:

```
driver.switch_to.default_content()
```

✓ **Best Practice with Explicit Wait**

Senior engineers avoid immediate switching.

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
WebDriverWait(driver, 10).until(
    EC.frame_to_be_available_and_switch_to_it((By.ID, "loginFrame"))
)
```

Then directly:

```
driver.find_element(By.ID, "username").send_keys("Admin")
```

Why better?

It waits until frame becomes available.

Avoids:

NoSuchFrameException

✓ **Reusable Framework Method**

Senior utility method.

```
def switch_to_frame(driver, locator, timeout=10):
    WebDriverWait(driver, timeout).until(
        EC.frame_to_be_available_and_switch_to_it(locator)
    )
```

Usage

```
switch_to_frame(driver, (By.ID, "paymentFrame"))
```

Return:

```
driver.switch_to.default_content()
```

✓ **Detect Number of Frames**

Useful in debugging.

```
frames = driver.find_elements(By.TAG_NAME, "iframe")
print("Total frames:", len(frames))
```

Example

```
for frame in frames:
    print(frame.get_attribute("id"))
```

✓ **Handle Unknown Dynamic Frame**

When frame id changes dynamically.

```
frames = driver.find_elements(By.TAG_NAME, "iframe")
```

for frame in frames:

```
driver.switch_to.frame(frame)
```

```
if driver.find_elements(By.ID, "targetElement"):
```

```
    driver.find_element(By.ID, "targetElement").click()
```

```
    break
```

```
driver.switch_to.default_content()
```

Senior approach:

Search frame dynamically.



Real Project Example

Payment gateway inside iframe.

```
WebDriverWait(driver, 15).until(
```

```
    EC.frame_to_be_available_and_switch_to_it((By.XPATH,
```

```
    "//iframe[contains(@src,'payment')]"))
```

```
)
```

```
driver.find_element(By.ID, "cardNumber").send_keys("123456789012")
```

```
driver.find_element(By.ID, "pay").click()
```

```
driver.switch_to.default_content()
```



Common Exceptions

Exception	Reason
NoSuchFrameException	frame not found
NoSuchElementException	forgot to switch
TimeoutException	frame load delayed



Common Mistakes



Wrong

```
driver.find_element(By.ID, "insideFrame")
```

Without switching.



Wrong

```
driver.switch_to.frame(5)
```

Invalid index.



Best Practices (Senior Level)

✓ Prefer WebElement or explicit wait

✓ Always switch back

✓ Handle nested frames carefully

- ✓ Use reusable utility
- ✓ Avoid hardcoded index when possible
- ✓ Validate frame loaded before action

✓ Framework Utility Class Example

class FrameHandler:

```
def __init__(self, driver):
    self.driver = driver

def switch_by_locator(self, locator, timeout=10):
    WebDriverWait(self.driver, timeout).until(
        EC.frame_to_be_available_and_switch_to_it(locator)
    )

def switch_default(self):
    self.driver.switch_to.default_content()

def switch_parent(self):
    self.driver.switch_to.parent_frame()
```

Usage

```
frame = FrameHandler(driver)
frame.switch_by_locator((By.ID, "loginFrame"))
frame.switch_default()
```

✓ Summary Table

Method	Purpose
switch_to.frame()	switch into frame
switch_to.default_content()	return main page
switch_to.parent_frame()	go one level up
frame_to_be_available_and_switch_to_it()	wait + switch

✓ Interview Answer

Frames and iframes are handled in Selenium by switching the driver context using `switch_to.frame()`. Frames can be identified by index, name, or WebElement, but in enterprise frameworks the preferred approach is using explicit waits with `frame_to_be_available_and_switch_to_it()` for reliability. After operations, the driver should always switch back using `default_content()` or `parent_frame()`.

If you'd like, I can explain **how to handle shadow DOM elements**, because many modern applications now use Shadow Root and it is often confused with iframe handling.

Handling **multiple windows and browser tabs** in Selenium is an important real-world automation topic because many applications open:

- child windows
 - payment popups
 - report windows
 - authentication pages
 - new browser tabs
-

In Selenium Python, this is managed using:

`driver.window_handles`

`driver.current_window_handle`

`driver.switch_to.window()`

Core Concepts

1. `driver.current_window_handle`

Returns the current active window ID.

```
parent_window = driver.current_window_handle
```

```
print(parent_window)
```

Example output:

```
CDwindow-5A9F1C1B
```

2. `driver.window_handles`

Returns list of all opened windows/tabs.

```
all_windows = driver.window_handles
```

```
print(all_windows)
```

Example:

```
['parent_id', 'child_id']
```

3. `driver.switch_to.window(handle)`

Used to move control to another tab/window.

```
driver.switch_to.window(child_window)
```

Difference Between Tab and Window

Selenium treats both the same way.

Both are handled using:

```
driver.window_handles
```

No separate method for tab vs window.

Basic Example – Switch to Child Window

Scenario:

Click button → child window opens → switch → validate → close → return parent

```
parent_window = driver.current_window_handle
```

```
driver.find_element(By.ID, "openWindow").click()
```

```
all_windows = driver.window_handles
```

```
for window in all_windows:
    if window != parent_window:
        driver.switch_to.window(window)
        break
```

```
print(driver.title)
```

```
driver.close()
```

```
driver.switch_to.window(parent_window)
```

✔ Step-by-Step Explanation

Store parent window

```
parent_window = driver.current_window_handle
```

Very important to return later.

Click to open new window

```
driver.find_element(By.ID, "openWindow").click()
```

Get all windows

```
all_windows = driver.window_handles
```

Find child window

```
for window in all_windows:
    if window != parent_window:
        driver.switch_to.window(window)
        break
```

Close child

```
driver.close()
```

Only closes current window.

Switch back parent

```
driver.switch_to.window(parent_window)
```

✔ Handling Multiple Child Windows

Suppose application opens many windows.

```
parent = driver.current_window_handle
```

```
driver.find_element(By.ID, "openMultiple").click()
```

```
for handle in driver.window_handles:
    if handle != parent:
        driver.switch_to.window(handle)
```



```
print(driver.title)
```

✅ **Close All Child Windows Only**

```
parent = driver.current_window_handle
```

```
for handle in driver.window_handles:
```

```
    if handle != parent:
```

```
        driver.switch_to.window(handle)
```

```
        driver.close()
```

```
driver.switch_to.window(parent)
```

✅ **Switch Based on Window Title**

Best practice in enterprise projects.

```
parent = driver.current_window_handle
```

```
driver.find_element(By.ID, "openReport").click()
```

```
for handle in driver.window_handles:
```

```
    driver.switch_to.window(handle)
```

```
    if "Report" in driver.title:
```

```
        print("Correct window found")
```

```
        break
```

✅ **Switch Based on URL**

Sometimes titles duplicate.

```
for handle in driver.window_handles:
```

```
    driver.switch_to.window(handle)
```

```
    if "invoice" in driver.current_url:
```

```
        break
```

✅ **Open New Tab in Selenium 4**

Selenium 4 supports opening new tabs directly.

New tab

```
driver.switch_to.new_window("tab")
```

```
driver.get("https://google.com")
```

New window

```
driver.switch_to.new_window("window")
```

```
driver.get("https://example.com")
```

✅ **Example with Explicit Wait**

Window may not open instantly.

Best practice:

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
parent = driver.current_window_handle
```

```
driver.find_element(By.ID, "openWindow").click()
```

```
WebDriverWait(driver, 10).until(
    EC.number_of_windows_to_be(2)
)
```

```
for handle in driver.window_handles:
    if handle != parent:
        driver.switch_to.window(handle)
        break
```

✅ **Reusable Framework Utility Method**

Senior framework style.

```
def switch_to_new_window(driver):
    parent = driver.current_window_handle
```

```
    WebDriverWait(driver, 10).until(
        lambda d: len(d.window_handles) > 1
    )
```

```
    for handle in driver.window_handles:
        if handle != parent:
            driver.switch_to.window(handle)
            return parent
```

Usage:

```
parent = switch_to_new_window(driver)
```

Return:

```
driver.switch_to.window(parent)
```

✅ **Validate Child Window**

```
assert "Payment" in driver.title
```

or

```
assert "payment" in driver.current_url
```

✅ **Common Mistakes**

 Wrong

driver.switch_to.window(1)
Window requires handle, not index.

 Wrong

driver.close()
driver.quit()
Calling both unnecessarily.

 Best Practices (Senior Level)

- ✓ Always save parent handle
 - ✓ Use explicit wait for new window
 - ✓ Switch using title/URL when possible
 - ✓ Close child window safely
 - ✓ Return to parent window
 - ✓ Wrap in utility methods
-

 Real Project Example

Payment gateway popup handling:
parent = driver.current_window_handle

driver.find_element(By.ID, "payNow").click()

WebDriverWait(driver, 10).until(
 EC.number_of_windows_to_be(2)
)

for handle in driver.window_handles:
 if handle != parent:
 driver.switch_to.window(handle)
 break

driver.find_element(By.ID, "cardNumber").send_keys("123456")
driver.find_element(By.ID, "submit").click()

driver.close()
driver.switch_to.window(parent)

 Summary Table

Method	Purpose
current_window_handle	current window
window_handles	all windows
switch_to.window()	switch window
close()	close current
new_window()	open new tab/window



Interview Answer

To handle multiple windows in Selenium, first store the parent window handle, trigger the new window, wait until the number of windows increases, iterate through window_handles, switch to the required child window, perform actions, close it, and switch back to the parent window. In framework design, this is usually wrapped in reusable utility methods for stability.

If you'd like, I can explain **how to handle dynamic unknown multiple windows in enterprise frameworks** where more than 3 windows can open randomly.

Handling **file uploads in Selenium Python** depends on how the application implements the upload control. In most enterprise automation projects, a senior engineer first checks whether the upload is:

1. **Standard HTML file input**
 2. **Custom upload button**
 3. **OS native popup**
-

Because each requires a different approach.

✅ **Most Common Method — send_keys()**

The best practice in Selenium is to send the file path directly to:

```
<input type="file">
```

Selenium bypasses the system file dialog.

✅ **Example HTML**

```
<input type="file" id="uploadFile">
```

✅ **Complete Script**

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
```

```
driver.get("https://example.com")
```

```
file_input = driver.find_element(By.ID, "uploadFile")
```

```
file_input.send_keys("C:/Users/Neeraj/Documents/sample.pdf")
```

```
driver.quit()
```

✅ **Explanation**

```
file_input.send_keys("C:/Users/Neeraj/Documents/sample.pdf")
```

Selenium directly uploads the file.

No need to click:

Choose File

popup.

✅ **Why send_keys() works**

Because browser file upload fields accept:

absolute file path

Selenium enters the path into the hidden input.

✅ **Use Absolute Path (Recommended)**

Use Python path handling.

```
import os
```

```
file_path = os.path.abspath("test_data/sample.pdf")
```

```
file_input.send_keys(file_path)
```

Better for framework portability.

✅ Example with Pytest Framework

```
def test_upload_file(driver):  
    driver.get("https://example.com")  
  
    file_path = os.path.abspath("files/test.pdf")  
  
    driver.find_element(By.ID, "uploadFile").send_keys(file_path)  
  
    driver.find_element(By.ID, "submit").click()
```

✅ Hidden File Input Handling

Sometimes file input is hidden.

Example:

```
<input type="file" style="display:none">
```

Normal Selenium may fail.

Use JavaScript to make visible

```
file_input = driver.find_element(By.ID, "uploadFile")
```

```
driver.execute_script(  
    "arguments[0].style.display='block';",  
    file_input  
)
```

```
file_input.send_keys(file_path)
```

Senior-level approach for hidden controls.

✅ Custom Upload Button Scenario

Sometimes UI shows button:

```
<button>Upload</button>
```

```
<input type="file" hidden>
```

Button opens OS popup.

Still use hidden input directly.

Example

```
hidden_input = driver.find_element(By.XPATH, "//*[@type='file']")  
hidden_input.send_keys(file_path)
```

No need to click button.

✅ Multiple File Upload

If control supports multiple files.

HTML:

```
<input type="file" multiple>
```

Upload multiple files

```
files = [  
    os.path.abspath("file1.pdf"),  
    os.path.abspath("file2.pdf")  
]  
  
driver.find_element(By.ID, "uploadFile").send_keys("\n".join(files))
```

Explanation

Files separated using:
newline

Verify Upload Success

Check uploaded filename.

```
uploaded = driver.find_element(By.ID, "uploadedFileName").text  
assert "sample.pdf" in uploaded
```

Wait Before Upload

Use explicit wait.

```
from selenium.webdriver.support.ui import WebDriverWait  
from selenium.webdriver.support import expected_conditions as EC  
  
upload = WebDriverWait(driver, 10).until(  
    EC.presence_of_element_located((By.ID, "uploadFile"))  
)  
  
upload.send_keys(file_path)
```

Prevents timing issues.

Reusable Utility Method

Senior framework design.

```
import os  
  
def upload_file(driver, locator, file_name):  
    file_path = os.path.abspath(file_name)  
    element = driver.find_element(*locator)  
    element.send_keys(file_path)
```

Usage

```
upload_file(driver, (By.ID, "uploadFile"), "files/test.pdf")
```

Handling OS Native Popup (Rare)

Some apps force native Windows popup.
Selenium cannot handle OS popup directly.
Possible tools:

- AutoIT (Windows)
- PyAutoGUI
- Robot Framework helpers

Example with PyAutoGUI

```
import pyautogui

driver.find_element(By.ID, "uploadBtn").click()

pyautogui.write(file_path)
pyautogui.press("enter")
```

But preferred approach:
Always try hidden input first.

 Real Project Example

```
Upload invoice.

def test_upload_invoice(driver):
    driver.get("https://example.com/invoice")

    path = os.path.abspath("documents/invoice.pdf")

    driver.find_element(By.ID, "invoiceUpload").send_keys(path)

    driver.find_element(By.ID, "saveBtn").click()
```

 Common Exceptions

Exception	Reason
InvalidArgumentException	wrong file path
ElementNotInteractableException	hidden input
NoSuchElementException	wrong locator

 Common Mistakes

 **Relative path only**
send_keys("test.pdf")
May fail.

 **Correct**
send_keys(os.path.abspath("test.pdf"))

 **Clicking upload button unnecessarily**

Often not needed.

 Best Practices (Senior Level)

- ✓ Use send_keys()
- ✓ Use absolute path
- ✓ Prefer hidden input directly
- ✓ Add explicit waits
- ✓ Verify upload success
- ✓ Wrap in utility method
- ✓ Avoid OS popup tools unless required

 Summary Table

Scenario	Best Solution
Standard input	send_keys()
Hidden input	JS + send_keys()
Multiple files	newline-separated paths
Native popup	PyAutoGUI / AutoIT

 Interview Answer

In Selenium Python, file uploads are typically handled by locating the input[type='file'] element and passing the absolute file path using send_keys(). For hidden controls, JavaScript may be used to expose the element first. In enterprise frameworks, file upload logic is usually implemented as a reusable utility method with validation after upload.

If you'd like, I can explain **how to upload files in remote Selenium Grid / Docker execution**, because that requires a slightly different approach in enterprise environments.

In Selenium Python, **JavaScriptExecutor** is used when normal Selenium methods cannot interact reliably with the browser.

Senior automation engineers use it for:

- hidden elements
- complex clicks
- scrolling
- DOM manipulation
- retrieving browser values
- handling disabled fields
- advanced synchronization

In Python, JavaScriptExecutor is available through:

driver.execute_script()

driver.execute_async_script()

Unlike Java, Python does not create a separate JavascriptExecutor object because every WebDriver already supports JavaScript execution.

 Why JavaScriptExecutor is Needed

Sometimes normal Selenium fails:

element.click()

Possible errors:

- ElementClickInterceptedException
- ElementNotInteractableException
- hidden element
- overlay blocking

Then JavaScript can directly interact with DOM.

 Main JavaScript Methods in Selenium Python

There are mainly **2 methods**

Method	Purpose
execute_script()	synchronous JavaScript
execute_async_script()	asynchronous JavaScript

 1. execute_script()

Runs JavaScript immediately.

Syntax

driver.execute_script(script, *args)

Arguments

Argument	Description
script	JavaScript code string
*args	WebElements or values passed to JS

Example

```
driver.execute_script("alert('Hello')")
```

Return value

JavaScript can return value.

```
title = driver.execute_script("return document.title;")
print(title)
```

Output:

Dashboard

 2. execute_async_script()

Used for async JavaScript.

Useful when JS needs callback.

Syntax

driver.execute_async_script(script, *args)

Example

```
driver.execute_async_script("""
    var callback = arguments[arguments.length - 1];
    window.setTimeout(function(){
        callback("Completed");
    }, 3000);
""")
```

Explanation

Last argument automatically becomes callback.

 Passing Arguments to JavaScript

You can pass WebElements.

Example

```
element = driver.find_element(By.ID, "username")
```

```
driver.execute_script(
    "arguments[0].value='Admin';",
    element
)
```

arguments[] mapping

Python arg JavaScript

first arg	arguments[0]
second arg	arguments[1]
third arg	arguments[2]

Example multiple args

```
driver.execute_script(  
    "arguments[0].style.border=arguments[1]",  
    element,  
    "3px solid red"  
)
```

 Common Uses in Real Projects

1. Click element using JS

Normal click fails.
element = driver.find_element(By.ID, "saveBtn")

```
driver.execute_script(  
    "arguments[0].click();",  
    element  
)
```

2. Scroll to element

```
driver.execute_script(  
    "arguments[0].scrollIntoView(true);",  
    element  
)
```

3. Scroll page down

```
driver.execute_script("window.scrollTo(0,500);")
```

4. Scroll to bottom

```
driver.execute_script(  
    "window.scrollTo(0, document.body.scrollHeight);"  
)
```

5. Get page title

```
title = driver.execute_script("return document.title;")
```

6. Get URL

```
url = driver.execute_script("return document.URL;")
```

7. Highlight element

Useful in debugging.
driver.execute_script(
 "arguments[0].style.border='3px solid red';",
 element
)

8. Set value directly

```
driver.execute_script(  
    "arguments[0].value='Neeraj';",  
    element  
)
```

```
"arguments[0].value='Neeraj';",  
    element  
)  
  
9. Remove readonly attribute  
driver.execute_script(  
    "arguments[0].removeAttribute('readonly');",  
    element  
)
```

10. Remove disabled attribute

```
driver.execute_script(  
    "arguments[0].removeAttribute('disabled');",  
    element  
)
```

11. Refresh page

```
driver.execute_script("history.go(0);")
```

12. Open new tab

```
driver.execute_script("window.open('https://google.c  
om');")
```

13. Get hidden text

```
text = driver.execute_script(  
    "return arguments[0].textContent;",  
    element  
)
```

14. Change background color

```
driver.execute_script(  
    "arguments[0].style.background='yellow';",  
    element  
)
```

15. Get inner HTML

```
html = driver.execute_script(  
    "return arguments[0].innerHTML;",  
    element  
)
```

 Advanced Wait with JavaScript

Wait until page fully loaded.

```
WebDriverWait(driver, 20).until(  
    lambda d: d.execute_script(  
        "return document.readyState"
```

```
) == "complete"
)
```

Why senior engineers use this

Ensures:
complete
before interacting.

✔ Return Types from JavaScript

Selenium converts JS return values.

JS Return	Python Type
string	str
number	int/float
boolean	bool
array	list
object	dict
element	WebElement

Example

```
result = driver.execute_script("return [1,2,3];")
print(result)
Output:
[1, 2, 3]
```

✔ Reusable Framework Utility

Senior framework helper.

```
class JavaScriptHelper:

    def __init__(self, driver):
        self.driver = driver

    def click(self, element):
        self.driver.execute_script(
            "arguments[0].click();", element
        )

    def scroll_into_view(self, element):
        self.driver.execute_script(
            "arguments[0].scrollIntoView(true);", element
        )

    def highlight(self, element):
        self.driver.execute_script(
            "arguments[0].style.border='2px solid red';",
```

```
element
)
```

```
def get_title(self):
    return self.driver.execute_script(
        "return document.title;"
    )
```

Usage

```
js = JavaScriptHelper(driver)
js.scroll_into_view(element)
js.click(element)
```

✔ Common Mistakes

✗ Wrong

```
driver.execute_script(arguments[0].click(), element)
```

Missing quotes around JS.

Correct

```
driver.execute_script("arguments[0].click();",
element)
```

✗ Overuse JavaScript

Do NOT replace all Selenium actions with JS.
Use only when normal Selenium fails.

✔ Best Practices (Senior Level)

- ✔ Use JS only when necessary
- ✔ Prefer explicit waits first
- ✔ Wrap JS in utility methods
- ✔ Log JS usage in framework
- ✔ Avoid bypassing application logic unnecessarily
- ✔ Use for scrolling and hidden elements

✔ Real Project Example

Date field readonly.

```
date_field = driver.find_element(By.ID, "date")

driver.execute_script(
    "arguments[0].removeAttribute('readonly');",
    date_field
)

date_field.send_keys("30-04-2026")
```

✔ Summary Table

Method	Purpose
execute_script()	run JS
execute_async_script()	async JS
arguments[0]	first passed argument
return	get value from JS

✔ Interview Answer

In Selenium Python, `JavaScriptExecutor` is implemented using `execute_script()` and `execute_async_script()`. It allows direct interaction with the browser DOM for operations such as clicking hidden elements, scrolling, modifying attributes, retrieving values, and waiting for page readiness. In enterprise frameworks, JavaScript execution is usually wrapped in reusable helper methods and used only when standard Selenium actions are insufficient.

If you'd like, I can explain how `JavaScriptExecutor` helps solve `ElementClickInterceptedException` and `StaleElementReferenceException` in real projects, since that is a common senior-level interview topic.

JAVASCRIPTEXECUTOR IN SELENIUM WITH JAVA REFRESH BROWSER <pre>public static void refreshBrowser(WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("history.go(0);") }</pre>	JAVASCRIPTEXECUTOR IN SELENIUM WITH PYTHON I'll convert each Java JavascriptExecutor utility into clean, reusable Python Selenium code , written the way you'd actually plug it into a pytest / framework utils file .
GET PAGE TEXT <pre>public static String getPageInnerTextByJs(WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; String pageText = js.executeScript("return document.documentElement.innerText;").toString(); return pageText; }</pre>	✓ Python Selenium – JavaScriptExecutor Utilities ✓ Works with: selenium.webdriver
SCROLL PAGES SCROLL BY PIXEL <pre>public static void scrollPageDown(WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("window.scrollTo(0,600);") }</pre>	◆ Common JS Utility Class (Python) from selenium.webdriver.common.by import By import time
SCROLL TO BOTTOM <pre>public static void scrollPageDown(WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("window.scrollTo(0,document.body.scrollHeight);") }</pre>	🔄 REFRESH BROWSER def refresh_browser(driver): driver.execute_script("history.go(0)") ✓ Alternative (preferred): def refresh_browser(driver): driver.refresh()
SCROLL INTO VIEW <pre>public static void scrollIntoView(WebElement element, WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("arguments[0].scrollIntoView(true);", element); }</pre>	📄 GET PAGE TEXT def get_page_inner_text(driver): return driver.execute_script("return document.documentElement.innerText;")
HIGHLIGHT ELEMENT <pre>public static void highlightElement(WebElement element, WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; String bgColor = element.getCssValue("backgroundColor"); for (int i = 0; i < 10; i++) { changeColor("rgb(0,200,0)", element, driver); changeColor(bgColor, element, driver); } }</pre>	📄 SCROLL PAGES ◆ Scroll by Pixel def scroll_by_pixel(driver, x=0, y=600): driver.execute_script(f"window.scrollTo({x}, {y})")
FLASHING ELEMENT USING COLOR CHANGE LOGIC <pre>public static void changeColor(String color, WebElement element, WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript(</pre>	◆ Scroll to Bottom def scroll_to_bottom(driver): driver.execute_script("window.scrollTo(0, document.body.scrollHeight)")
DRAW BORDER FOR ELEMENT <pre>def draw_border(driver, element, color="red"): driver.execute_script(f"arguments[0].style.border='3px solid {color}';", element)</pre>	◆ Scroll Into View def scroll_into_view(driver, element): driver.execute_script("arguments[0].scrollIntoView(true);", element)
FLASHING ELEMENT USING COLOR CHANGE LOGIC <pre>public static void changeColor(String color, WebElement element, WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript(</pre>	🌟 HIGHLIGHT ELEMENT def highlight_element(driver, element, loops=10): original_bg = element.value_of_css_property("background-color") for _ in range(loops): change_color(driver, element, "rgb(0,200,0)") time.sleep(0.02) change_color(driver, element, original_bg)
FLASHING ELEMENT USING COLOR CHANGE LOGIC <pre>public static void changeColor(String color, WebElement element, WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript(</pre>	🔄 FLASH ELEMENT – COLOR CHANGE def change_color(driver, element, color): driver.execute_script(f"arguments[0].style.backgroundColor = '{color}';", element)
FLASHING ELEMENT USING COLOR CHANGE LOGIC <pre>public static void changeColor(String color, WebElement element, WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript(</pre>	📄 DRAW BORDER FOR ELEMENT def draw_border(driver, element, color="red"): driver.execute_script(f"arguments[0].style.border='3px solid {color}';", element) ✓ Thin border:

<pre>"arguments[0].style.backgroundColor = "" + color + """, element); try { Thread.sleep(20); } catch (InterruptedException e) { } DRAW BORDER FOR AN ELEMENT public static void drawBorder(WebElement element, WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("arguments[0].style.border='2px solid red'", element); }</pre>	<pre>def draw_border(driver, element): driver.execute_script("arguments[0].style.border='2px solid red'", element)</pre>
	 GENERATE ALERT <pre>def generate_alert(driver, message): driver.execute_script(f"alert('{message}')</pre>
	 CLICK ELEMENT USING JS <pre>def click_by_js(driver, element): driver.execute_script("arguments[0].click();", element)</pre> ✔️ Use when ElementClickInterceptedException happens.
	 GET PAGE TITLE <pre>def get_page_title(driver): return driver.execute_script("return document.title;")</pre>
	✔️ How to Use in Test / Framework <pre>def test_example(driver): element = driver.find_element(By.ID, "submit") scroll_into_view(driver, element) highlight_element(driver, element) click_by_js(driver, element) assert "Dashboard" in get_page_title(driver)</pre>
<pre>GENERATE ALERT public static void generateAlert(WebDriver driver, String message) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("alert("" + message + "")"); }</pre>	✔️ Best Practice Notes (Important ✔️) ✔️ Prefer normal Selenium first ✔️ Use JS when: • Element not clickable • Custom scrolling needed • UI animations break normal actions • Debugging (highlight, border) ✔️ Never overuse JS clicks — it bypasses real user behavior
<pre>CLICK ON AN ELEMENT public static void clickEleByJs(WebElement element, WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("arguments[0].click();", element); }</pre>	If you want next: • ✔️ Merge this into your existing CommonMethod class • ✔️ Add logging + screenshots • ✔️ Convert to Playwright JS • ✔️ Make it fixture-based with pytest Just tell me 👍
<pre>GET PAGE TITLE public static String getTitleByJs(WebDriver driver) { JavascriptExecutor js = (JavascriptExecutor) driver; String title = js.executeScript("return document.title;").toString(); return title; }</pre>	

Taking screenshots in **Selenium Python** is a common requirement in automation frameworks for:

- failure analysis
- test reporting
- debugging
- evidence capture
- CI/CD execution logs

A senior automation engineer usually implements screenshots in a **reusable utility method** rather than writing raw code in every test.

 Ways to take screenshot in Selenium

Selenium provides multiple methods.

Method	Purpose
save_screenshot()	save image file
get_screenshot_as_file()	save image file
get_screenshot_as_png()	return bytes
get_screenshot_as_base64()	return base64
element.screenshot()	specific element screenshot

 1. Full Page Screenshot (Basic)

Complete Script

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

driver.get("https://example.com")
driver.maximize_window()

driver.save_screenshot("homepage.png")

driver.quit()
```

Explanation

driver.save_screenshot("homepage.png")
This saves screenshot in current project folder.

 2. Using get_screenshot_as_file()

Same purpose.

```
driver.get_screenshot_as_file("page.png")
```

Complete Script

```
from selenium import webdriver
```



```
driver = webdriver.Chrome()
driver.get("https://example.com")

driver.get_screenshot_as_file("page.png")

driver.quit()
```

✅ 3. Screenshot with Timestamp (Best Practice)

Avoid overwriting old files.

```
from selenium import webdriver
from datetime import datetime

driver = webdriver.Chrome()
driver.get("https://example.com")

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
file_name = f"screenshot_{timestamp}.png"

driver.save_screenshot(file_name)

driver.quit()
```

Example file:
screenshot_20260430_103522.png

✅ 4. Save Screenshot in Folder

Professional framework style.

```
import os
from datetime import datetime

folder = "screenshots"
os.makedirs(folder, exist_ok=True)

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
path = os.path.join(folder, f"error_{timestamp}.png")

driver.save_screenshot(path)
```

Folder structure

```
project/
├─ tests/
├─ screenshots/
└─ error_20260430.png
```

✅ 5. Screenshot of Specific Element

Only capture one element.

```
logo = driver.find_element(By.ID, "logo")
logo.screenshot("logo.png")
```

Complete Example

```
from selenium import webdriver
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
driver.get("https://example.com")
```

```
element = driver.find_element(By.TAG_NAME, "h1")
element.screenshot("heading.png")
```

```
driver.quit()
```

Useful for:

- ✓ logos
 - ✓ charts
 - ✓ error messages
 - ✓ reports
-

✓ 6. Screenshot as PNG Bytes

Useful for reporting.

```
png_data = driver.get_screenshot_as_png()
Returns binary data.
```

Example

```
with open("test.png", "wb") as file:
    file.write(driver.get_screenshot_as_png())
```

✓ 7. Screenshot as Base64

Useful for HTML reports.

```
base64_image = driver.get_screenshot_as_base64()
Used in:
```

- Allure
 - pytest-html
 - custom reporting
-

✓ Complete Reusable Utility Method

Senior framework approach.

```
import os
from datetime import datetime
```

```
def take_screenshot(driver, test_name):
    folder = "screenshots"
    os.makedirs(folder, exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    path = os.path.join(folder, f"{test_name}_{timestamp}.png")

    driver.save_screenshot(path)

    return path
```

Usage in test

```
def test_login(driver):
    driver.get("https://example.com")

    path = take_screenshot(driver, "test_login")
    print("Screenshot saved:", path)
```



Screenshot on Failure in Pytest

Professional automation framework.

Hook Example

```
import pytest

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item):
    outcome = yield
    report = outcome.get_result()

    if report.when == "call" and report.failed:
        driver = item.funcargs["driver"]
        take_screenshot(driver, item.name)
```

Automatically captures only when test fails.



Full Working Example

Complete script.

```
from selenium import webdriver
from datetime import datetime
import os

def take_screenshot(driver):
    folder = "screenshots"
    os.makedirs(folder, exist_ok=True)
```

```
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
path = os.path.join(folder, f"screenshot_{timestamp}.png")
```

```
driver.save_screenshot(path)
print("Screenshot saved:", path)
```

```
driver = webdriver.Chrome()
driver.maximize_window()
```

```
driver.get("https://example.com")
```

```
take_screenshot(driver)
```

```
driver.quit()
```

✔ Common Mistakes

✗ Wrong path

```
driver.save_screenshot("")
```

Invalid filename.

✗ Taking screenshot before page load

Better:

```
WebDriverWait(driver, 10).until(
    lambda d: d.execute_script("return document.readyState") == "complete"
)
Then screenshot.
```

✔ Best Practices (Senior Level)

- ✔ Use timestamp
- ✔ Save in dedicated folder
- ✔ Capture on failure only
- ✔ Use reusable utility
- ✔ Attach to reports
- ✔ Capture element-level when needed

✔ Real Project Example

Capture failure after exception.

try:

```
driver.find_element(By.ID, "submit").click()
```

except Exception:

```
take_screenshot(driver, "submit_error")
raise
```

✔ Summary Table

Method	Use
save_screenshot()	save file
get_screenshot_as_file()	save file
get_screenshot_as_png()	bytes
get_screenshot_as_base64()	base64
element.screenshot()	single element



Interview Answer

In Selenium Python, screenshots can be captured using `driver.save_screenshot()` or `driver.get_screenshot_as_file()`. In enterprise frameworks, screenshots are usually stored with timestamps inside a reusable utility method and automatically attached to reports whenever a test fails.

If you'd like, I can explain **how to attach Selenium screenshots into pytest-html report automatically**, which is very useful in framework interviews.

To validate **broken links in Selenium**, the standard senior-level approach is:

1. Use Selenium to collect all links from the page
2. Extract each URL from the href attribute
3. Send an HTTP request to each URL
4. Check the response status code
5. Identify broken links (usually 404, 500, etc.)

Selenium itself cannot verify HTTP response codes, so in Python we combine Selenium with: requests

 What is a broken link?

A broken link usually returns:

Status Code	Meaning
404	Not Found
500	Server Error
403	Forbidden
400	Bad Request

Healthy links usually return:
200

 Complete Script to Validate Broken Links

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import requests
```

```
driver = webdriver.Chrome()
driver.get("https://example.com")
```

```
links = driver.find_elements(By.TAG_NAME, "a")
```

```
for link in links:
    url = link.get_attribute("href")
```

```
if url:
    response = requests.get(url)
```

```
    if response.status_code >= 400:
        print(f"Broken link: {url} | Status:
{response.status_code}")
    else:
        print(f"Valid link: {url} | Status:
{response.status_code}")
```

```
driver.quit()
```

 Step-by-step Explanation

1. Find all anchor tags

```
links = driver.find_elements(By.TAG_NAME, "a")
```

Collects all:

elements.

2. Extract href

```
url = link.get_attribute("href")
```

Gets actual URL.

Example:
`https://example.com/contact`

3. Send HTTP request

```
response = requests.get(url)
```

4. Check status code

```
if response.status_code >= 400:
```

Broken links typically:

- 404
- 500
- 403

 Better Script with Exception Handling

Production-grade version.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import requests
```

```
driver = webdriver.Chrome()
driver.get("https://example.com")
```

```
links = driver.find_elements(By.TAG_NAME, "a")
```

```
for link in links:
    url = link.get_attribute("href")
```

```
    if not url:
        continue
```

```
    try:
        response = requests.get(url, timeout=5)
```

```
        if response.status_code >= 400:
            print(f"Broken: {url} -> {response.status_code}")
        else:
            print(f"Valid: {url} -> {response.status_code}")
```

```
    except requests.exceptions.RequestException as e:
        print(f"Error checking {url}: {e}")
```

```
driver.quit()
```

Why exception handling?

Some links may:

• timeout • refuse connection • redirect badly
Without handling script may stop.
✔ Using HEAD request (Faster) More efficient for large sites. response = requests.head(url, allow_redirects=True)
Example response = requests.head(url, timeout=5, allow_redirects=True) HEAD avoids downloading full page.
✔ Recommended Senior Version response = requests.head(url, timeout=5, allow_redirects=True)
✔ Validate Image Links Too For images: images = driver.find_elements(By.TAG_NAME, "img") for img in images: src = img.get_attribute("src") Check image URLs the same way.
✔ Reusable Framework Method Best practice utility. import requests def check_broken_link(url): try: response = requests.head(url, timeout=5, allow_redirects=True) if response.status_code >= 400: return False, response.status_code return True, response.status_code except Exception: return False, "Exception"
Usage status, code = check_broken_link(url)
✔ Full Framework Example def validate_all_links(driver): links = driver.find_elements(By.TAG_NAME, "a") for link in links:

url = link.get_attribute("href") if url: valid, code = check_broken_link(url) if not valid: print(f"Broken link found: {url} ({code})")
✔ Store Broken Links in List Useful for reports. broken_links = [] for link in links: url = link.get_attribute("href") if url: response = requests.head(url) if response.status_code >= 400: broken_links.append(url)
Print summary print("Total broken links:", len(broken_links))
✔ Validate Only Internal Links Avoid checking external sites. if "example.com" in url: Example: if url.startswith("https://example.com"):
✔ Common Issues
Skip empty links if url is None: continue
Skip JavaScript links if "javascript" in url.lower(): continue
Skip mail links if url.startswith("mailto"): continue
Senior filtering example if not url or url.startswith("javascript") or url.startswith("mailto"): continue
✔ Real Project Example

```
driver.get("https://companyportal.com")
```

```
validate_all_links(driver)
```

✔ Common Mistakes

✗ Clicking every link

Bad approach:
link.click()
Very slow.

✔ Better

Use HTTP request.

✗ Using GET for all pages

Slower.

✔ Better

requests.head()

✔ Best Practices (Senior Level)

- ✔ Use HEAD request
- ✔ Handle redirects
- ✔ Skip mailto/javascript
- ✔ Capture broken links in report
- ✔ Validate internal links first
- ✔ Build utility reusable method

✔ Summary Table

Step	Action
Find links	find_elements(By.TAG_NAME,"a")
Get URL	get_attribute("href")
Send request	requests.head()
Check status	status_code >= 400

✔ Interview Answer

To validate broken links in Selenium, I first collect all anchor elements using Selenium, extract their href values, and then use Python’s requests library to send an HTTP HEAD request. If the response status code is 400 or above, the link is considered broken. In automation frameworks, this logic is typically implemented as a reusable utility that logs broken URLs for reporting.

If you'd like, I can show you how to fail a pytest test automatically when broken links are found, which is useful in framework interviews.

CDP and Network log

Capturing **network logs in Selenium Python** is an advanced topic often used in automation frameworks to inspect:

- API requests
- response status codes
- failed resources
- authentication headers
- performance issues

A senior automation engineer typically captures network logs using:

1. **Chrome DevTools Protocol (CDP)** — recommended
2. Browser performance logs
3. Selenium 4 DevTools integration concepts

In Python, the most practical approach is using **Chrome performance logging**.

Why capture network logs?

Useful to validate:

- ✓ API called correctly
- ✓ Response code is 200
- ✓ No failed requests
- ✓ Backend call triggered
- ✓ Slow resources
- ✓ Security headers

Method 1 — Using Chrome Performance Logs

This is the most common Selenium-based approach.

Complete Script

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import json
```

```
options = webdriver.ChromeOptions()
```

```
options.set_capability(
    "goog:loggingPrefs",
    {"performance": "ALL"}
)
```

```
driver = webdriver.Chrome(options=options)
```

```
driver.get("https://example.com")
```

```
logs = driver.get_log("performance")
```

for entry in logs:

```
    log = json.loads(entry["message"])
    message = log["message"]
```

```
    if message["method"] ==
"Network.responseReceived":
```

```
        url = message["params"]["response"]["url"]
        status = message["params"]["response"]["status"]
```

```
        print(f"URL: {url}")
        print(f"Status: {status}")
        print("-" * 50)
```

```
driver.quit()
```

Explanation

Enable logging

```
options.set_capability(
    "goog:loggingPrefs",
    {"performance": "ALL"}
)
```

This enables browser network logs.

Get logs

```
logs = driver.get_log("performance")
```

Returns all network/performance events.

Parse JSON

```
log = json.loads(entry["message"])
```

Each log entry is JSON data.

Filter network responses

```
if message["method"] ==
"Network.responseReceived":
```

Only response events.

Example Output

URL: https://example.com/api/login
Status: 200

URL: https://example.com/logo.png
Status: 200

Method 2 — Capture Only Failed Requests

Useful in enterprise validation.

```
for entry in logs:
    log = json.loads(entry["message"])
    message = log["message"]

    if message["method"] ==
"Network.responseReceived":
        response = message["params"]["response"]

    if response["status"] >= 400:
        print("Failed Request:", response["url"])
        print("Status:", response["status"])
```

Example output

Failed Request: https://example.com/api/user
Status: 500

Method 3 — Capture Request URLs Only

```
for entry in logs:
    log = json.loads(entry["message"])
    message = log["message"]

    if message["method"] ==
"Network.requestWillBeSent":
        request_url = message["params"]["request"]["url"]
        print(request_url)
```

Useful for verifying API calls.

Validate Specific API Call

Senior framework example.

```
target_api = "/api/login"

for entry in logs:
    log = json.loads(entry["message"])
    message = log["message"]

    if message["method"] ==
"Network.responseReceived":
        response = message["params"]["response"]

    if target_api in response["url"]:
        assert response["status"] == 200
```

Good for UI + API validation.

Reusable Utility Function

Framework style.

```
import json

def get_network_logs(driver):
    logs = driver.get_log("performance")
    network_data = []

    for entry in logs:
        log = json.loads(entry["message"])
        message = log["message"]

        if message["method"] ==
"Network.responseReceived":
            response = message["params"]["response"]

            network_data.append({
                "url": response["url"],
                "status": response["status"]
            })

    return network_data
```

Usage

network_logs = get_network_logs(driver)

```
for item in network_logs:
    print(item)
```

Output example

{'url': 'https://example.com/api/login', 'status': 200}

Validate API Response in Test

```
def test_api_status(driver):
    driver.get("https://example.com")

    logs = get_network_logs(driver)

    for item in logs:
        if "/api/login" in item["url"]:
            assert item["status"] == 200
```

Capture Headers

Advanced use.

```
headers = message["params"]["response"]["headers"]
print(headers)
```

Useful for:

- auth tokens
- cache headers
- content type

✔ Capture Request Method

```
request_method =
message["params"]["request"]["method"]
```

Possible output:

GET
POST
PUT
DELETE

✔ Common Problems

driver.get_log("performance") empty

Cause:
Logging not enabled.
Need:
options.set_capability("goog:loggingPrefs",
{ "performance": "ALL" })

✔ Browser Support

Browser Support

Chrome	Yes
Edge	Yes
Firefox	Limited

Chrome works best.

✔ Best Practices (Senior Level)

- ✓ Enable logging before browser launch
- ✓ Parse only required events
- ✓ Validate critical APIs
- ✓ Store logs in framework reports
- ✓ Use reusable utility methods
- ✓ Capture failed requests only when needed

✔ Real Project Example

Validate invoice API.

```
driver.find_element(By.ID, "saveInvoice").click()

logs = get_network_logs(driver)

for item in logs:
    if "/api/saveInvoice" in item["url"]:
        assert item["status"] == 200
```

Useful because:

UI may show success even if backend fails.

✔ Summary Table

Method	Purpose
driver.get_log("performance")	get logs
Network.requestWillBeSent	request data
Network.responseReceived	response data
response.status	status code
response.url	requested URL

✔ Interview Answer

In Selenium Python, network logs can be captured by enabling Chrome performance logging through browser capabilities and then reading logs using driver.get_log("performance"). The returned JSON events can be filtered for Network.requestWillBeSent and Network.responseReceived to validate API calls, response status codes, and failed resources. In enterprise frameworks, this is usually wrapped inside reusable utilities for backend validation during UI automation.

If you'd like, I can explain **how to capture request and response body payloads in Selenium using Chrome DevTools Protocol**, which is more advanced than normal network logs.

StaleElementReferenceException is one of the most common issues in Selenium automation. It happens when a **previously located WebElement is no longer attached to the DOM**.

What is StaleElementReferenceException?
It means:
“The element you are trying to interact with is no longer valid.”

When does it occur?
Typical real-world scenarios:

- Page refresh
- DOM updated via AJAX
- React / Angular re-render
- Navigation to another page
- Element recreated after action (like click/save)

Example of failure
element = driver.find_element(By.ID, "saveBtn")
driver.refresh()
element.click() # **StaleElementReferenceException**
Because after refresh:
👉 Old reference is invalid

Core Idea to Fix
👉 **Never reuse old WebElement after DOM change**
👉 Always **re-locate element**

Solution 1: Re-locate Element (Most Important)

```
driver.refresh()
element = driver.find_element(By.ID, "saveBtn")
element.click()
```

Why it works
Because you are getting a **fresh reference**

Solution 2: Use Explicit Wait with refreshed

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
element = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.ID, "saveBtn"))
)
element.click()
```

Better approach using refreshed

```
element = WebDriverWait(driver, 10).until(
    EC.refreshed(
        EC.presence_of_element_located((By.ID, "saveBtn"))
    )
)
element.click()
```

Solution 3: Try-Catch with Retry Logic
Useful in dynamic apps.
from selenium.common.exceptions import StaleElementReferenceException

```
for i in range(3):
    try:
        element = driver.find_element(By.ID, "saveBtn")
        element.click()
        break
    except StaleElementReferenceException:
        print("Retrying due to stale element...")
```

Why this works
Retries until DOM stabilizes.

Solution 4: Use Locator Instead of WebElement
Bad practice:
element = driver.find_element(By.ID, "btn")
Better:
locator = (By.ID, "btn")

```
driver.find_element(*locator).click()
```

Why?
Locator always fetches fresh element.

Solution 5: Wait for DOM to Stabilize

```
WebDriverWait(driver, 10).until(
    lambda d: d.execute_script("return document.readyState") == "complete"
)
```

Useful after navigation or refresh.

Solution 6: Handle After AJAX Update

```
from selenium.webdriver.support import expected_conditions as EC
old_element = driver.find_element(By.ID, "status")
WebDriverWait(driver, 10).until(
    EC.staleness_of(old_element)
)
new_element = driver.find_element(By.ID, "status")
print(new_element.text)
```

Why important?
Waits until old element disappears.

Solution 7: Page Object Model Best Practice
In POM:
✗ Avoid storing WebElement like:
self.button = driver.find_element(...)
✓ Instead:
def get_button(self):
 return self.driver.find_element(By.ID, "btn")

Usage
self.get_button().click()

Always fresh reference.
Solution 8: Advanced Retry Utility (Framework Style)

```
def click_with_retry(driver, locator, retries=3):
    for i in range(retries):
        try:
            driver.find_element(*locator).click()
            return
        except StaleElementReferenceException:
            if i == retries - 1:
                raise
```

Usage
click_with_retry(driver, (By.ID, "saveBtn"))

Common Mistakes

✗ Reusing old element
element.click()
driver.refresh()
element.click() # **✗**

✗ Using sleep blindly
time.sleep(5)
Does not guarantee fix.

Best Practices (Senior Level)
✓ Always re-locate elements
✓ Use explicit waits
✓ Use staleness_of() for dynamic UI
✓ Avoid storing WebElement in class variables
✓ Implement retry logic
✓ Prefer locator-based actions

Summary
StaleElementReferenceException occurs when the DOM changes and the element reference becomes invalid. It is handled by re-locating elements, using explicit waits like staleness_of or refreshed, and implementing retry logic in dynamic applications.

Interview Answer
StaleElementReferenceException occurs when a previously located element is no longer attached to the DOM due to page refresh or dynamic updates. It can be handled by re-locating the element, using explicit waits such as staleness_of or refreshed, and implementing retry mechanisms. In frameworks, we avoid storing WebElements and instead use locators to always fetch fresh elements.

ElementClickInterceptedException occurs when Selenium tries to click an element, but something else is blocking it.

 **Example Failure**
element = driver.find_element(By.ID, "submit")
element.click() #  ElementClickInterceptedException

 **Solution Strategy (Senior Approach)**
You don't just "fix it with JS click" — you identify the cause and handle it properly.

 1. Wait Until Element is Clickable (Most Important)

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

element = WebDriverWait(driver, 10).until(
    EC.element_to_be_clickable((By.ID, "submit"))
)
element.click()
```

Why it works

- ✓ waits for visibility
- ✓ waits for enable state
- ✓ avoids early click

 2. Wait for Overlay / Loader to Disappear
Very common in enterprise apps.

```
WebDriverWait(driver, 20).until(
    EC.invisibility_of_element_located((By.ID, "loader"))
)
driver.find_element(By.ID, "submit").click()
```

 3. Scroll Element Into View
If element is outside viewport.

```
element = driver.find_element(By.ID, "submit")
driver.execute_script(
    "arguments[0].scrollIntoView(true);", element
)
element.click()
```

Better version with wait

```
element = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.ID, "submit")))
driver.execute_script("arguments[0].scrollIntoView(true);", element)
WebDriverWait(driver, 5).until(EC.element_to_be_clickable((By.ID,
"submit"))))
element.click()
```

 4. Handle Popup / Modal Blocking Click

```
try:
    popup_close = driver.find_element(By.ID, "closePopup")
    popup_close.click()
except:
    pass
driver.find_element(By.ID, "submit").click()
```

 5. Click Using JavaScript (Fallback)
Use only if normal click fails.

```
element = driver.find_element(By.ID, "submit")
driver.execute_script(
    "arguments[0].click();", element
)
```

Why use carefully?

- ✓ bypasses UI
-  may skip real user behavior

 7. Move to Element Before Click
Sometimes hover required.

```
from selenium.webdriver.common.action_chains import ActionChains
element = driver.find_element(By.ID, "submit")
ActionChains(driver).move_to_element(element).click().perform()
```

Helps when element is covered or requires hover.

 Real Project Example

```
def test_submit_form(driver):
    driver.get("https://example.com")
    # wait for loader to disappear
    WebDriverWait(driver, 20).until(
        EC.invisibility_of_element_located((By.CLASS_NAME, "loading"))
    )

    # scroll to button
    element = driver.find_element(By.ID, "submit")
    driver.execute_script("arguments[0].scrollIntoView(true);", element)

    # click with retry
    for i in range(3):
        try:
            element = WebDriverWait(driver, 10).until(
                EC.element_to_be_clickable((By.ID, "submit"))
            )
            element.click()
            break
        except ElementClickInterceptedException:
            print("Retrying click...")
```

 Common Mistakes

 **Direct click without wait**
driver.find_element(By.ID, "submit").click()

 **Using JS click always**
Bad practice.

 **Ignoring overlays**

 When to Use What

Problem	Solution
Loader blocking	invisibility wait
Element not visible	scroll
Popup blocking	close popup
Animation delay	explicit wait
Still failing	JS click

 **Summary**
ElementClickInterceptedException occurs when another element blocks the target element. It is handled using explicit waits, scrolling, handling overlays or popups, retry logic, and as a last resort, JavaScript click.

 **Interview Answer**
ElementClickInterceptedException occurs when an element is present but another element overlaps it. It can be handled by waiting for the element to be clickable, ensuring overlays or loaders are gone, scrolling into view, and using retry logic. In rare cases, JavaScript click is used as a fallback in automation frameworks.

If you want, I can show you real debugging techniques to identify which element is intercepting the click using DevTools, which is a very strong senior-level skill.

1. NoSuchElementException

What it means

Selenium is unable to find the element in the DOM.

Example

element = driver.find_element(By.ID, "username") # Not found

Common Causes

- Wrong locator
- Element not loaded yet
- Inside iframe (not switched)
- Page not fully loaded
- Dynamic element (AJAX)

How to Handle

Solution 1: Use Explicit Wait (Best Practice)

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
element = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.ID, "username")))
element.send_keys("Admin")
```

Solution 2: Check inside iframe

```
driver.switch_to.frame("frameId")
driver.find_element(By.ID, "username").send_keys("Admin")
driver.switch_to.default_content()
```

Solution 3: Validate locator

```
elements = driver.find_elements(By.ID, "username")
if elements:
    elements[0].send_keys("Admin")
else:
    print("Element not found")
```

Solution 4: Wait for page load

```
WebDriverWait(driver, 10).until(
    lambda d: d.execute_script("return document.readyState") == "complete")
```

2. ElementNotInteractableException

What it means

Element is present in DOM but cannot be interacted with.

Example

```
element = driver.find_element(By.ID, "submit")
element.click() # Not interactable
```

Common Causes

- Element is hidden (display:none)
- Disabled (disabled=true)
- Covered by another element
- Not yet visible
- Inside collapsed section

How to Handle

Solution 1: Wait until clickable

```
element = WebDriverWait(driver, 10).until(
    EC.element_to_be_clickable((By.ID, "submit")))
element.click()
```

Solution 2: Scroll into view

```
element = driver.find_element(By.ID, "submit")
```

```
driver.execute_script(
    "arguments[0].scrollIntoView(true);",
    element)
element.click()
```

Solution 3: Remove disabled attribute (JS)

```
driver.execute_script(
    "arguments[0].removeAttribute('disabled');",
    element)
)
```

Solution 4: Wait for visibility

```
element = WebDriverWait(driver, 10).until(
```

```
EC.visibility_of_element_located((By.ID, "submit")))
```

Solution 5: JS Click (fallback)

```
driver.execute_script("arguments[0].click();", element)
```

3. ElementNotVisibleException

Note: This is deprecated in Selenium 4 and replaced by: ElementNotInteractableException

What it means

Element exists in DOM but is not visible.

Example

```
element = driver.find_element(By.ID, "hiddenBtn")
element.click() # Not visible
```

Common Causes

- CSS display: none
- visibility: hidden
- Off-screen
- Behind another element
- Hidden in modal/tab

How to Handle

Solution 1: Wait for visibility

```
element = WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.ID, "hiddenBtn")))
element.click()
```

Solution 2: Scroll

```
driver.execute_script(
    "arguments[0].scrollIntoView(true);",
    element)
```

Solution 3: Make visible using JS

```
driver.execute_script(
    "arguments[0].style.display='block';",
    element)
)
```

Solution 4: Click using JS

```
driver.execute_script(
    "arguments[0].click();",
    element)
)
```

Real Project Example (Combined Handling)

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
def safe_click(driver, locator, timeout=10):
    try:
        element = WebDriverWait(driver, timeout).until(
            EC.element_to_be_clickable(locator) )
        element.click()
    except Exception:
        element = driver.find_element(*locator)
        driver.execute_script("arguments[0].click();", element)
```

Usage

```
safe_click(driver, (By.ID, "submit"))
```

Comparison Table

Exception	Meaning	Fix
NoSuchElementException	element not found	wait / correct locator
ElementNotInteractableException	not clickable	wait / scroll
ElementNotVisibleException	hidden element	visibility wait

Interview Answer

NoSuchElementException occurs when Selenium cannot find an element, usually due to incorrect locator or timing issues. ElementNotInteractableException occurs when the element is present but not ready for interaction, often due to visibility or state issues. ElementNotVisibleException (deprecated) occurs when the element is hidden. These are handled using explicit waits, correct locators, scrolling, iframe handling, and JavaScript as a fallback.

If you want, I can give you a real interview scenario where all 3 exceptions happen together and how to debug them step-by-step, which is a strong senior-level answer.